

ESSENTIAL RESERVOIR COMPUTING

DISSERTATION

Presented in Partial Fulfillment of the Requirements for the Degree Doctor of
Philosophy in the Graduate School of The Ohio State University

By

Aaron Griffith, B.S.

Graduate Program in Physics

The Ohio State University

2021

Dissertation Committee:

Daniel J. Gauthier, Advisor

Amy Connolly

Ciriyam Jayaprakash

Gregory Lafyatis

© Copyright by
Aaron Griffith
2021

ABSTRACT

Reservoir computing (RC) is a machine learning method especially well suited to solving physical problems, by using an internal dynamic system known as a 'reservoir'. Many systems are suitable for use as an internal reservoir. A common choice is an *echo state network* (ESN), a network with recurrent connections that gives the RC a memory which it uses to efficiently solve many time-domain problems such as forecasting chaotic systems and hidden state inference. However, constructing an ESN involves a large number of poorly-understood meta-parameters, and the properties that an ESN must have to solve these tasks well are largely unknown.

In this dissertation, I explore what parts of an RC are absolutely necessary. I build ESNs that perform well at system forecasting despite an extremely simple internal network structure, without any recurrent connections at all, breaking one of the most common rules of ESN design. These simple reservoirs indicate that the role of the reservoir in the RC is only to remember a finite number of time-delays of the RC's input, and while a complicated network can achieve this, in many cases a simple one achieves this as well.

I then build upon a recent proof of the equivalence between a specific ESN construction and the nonlinear vector auto-regression (NVAR) method with my collaborators. The NVAR is an RC boiled down to its most essential components, taking the necessary time-delay taps directly rather than relying on an internal dynamic reservoir. I demonstrate these RCs-without-reservoirs on a variety of classical RC problems, showing that in many cases an NVAR will perform as well or better than an RC despite the simpler method. I then conclude with an example problem that highlights a remaining unsolved issue in the application of NVARs, and then look to a possible future where NVARs may supplant RCs.

For my brother Nathan, and my parents Gregory and Mary Lea.

ACKNOWLEDGMENTS

This work would not have been possible without the committed support of my advisor, Daniel J. Gauthier, or my insightful collaborators, including Wendson A. S. Barbosa, Erik Bollt, Daniel Canaday, and Andrew Pomerance.

In addition, many discussions over the years have re-contextualized my understanding of reservoir computers. I would specifically like to thank Michelle Girvan, Alexander Hartemink, Brian Hunt, Zhixin Lu, Edward Ott, and Jaideep Pathak.

VITA

2009-2014	B.S., Mathematics and Physics The Ohio State University
2015-2016	Graduate Teaching Associate Department of Physics The Ohio State University
2016-2021	Graduate Research Associate Department of Physics The Ohio State University

Publications

D. Canaday, A. Griffith, and D. J. Gauthier, “Rapid time series prediction with a hardware-based reservoir computer,” *Chaos: An Interdisciplinary Journal of Nonlinear Science* **28**, 123119 (2018).

A. Griffith, A. Pomerance, and D. J. Gauthier, “Forecasting chaotic systems with very low connectivity reservoir computers,” *Chaos: An Interdisciplinary Journal of Nonlinear Science* **29**, 123108 (2019).

Fields of Study

Major Field: Physics

Contents

	Page
Abstract	ii
Dedication	iii
Acknowledgments	iv
Vita	v
List of Figures	viii
List of Tables	xii

Chapters

1 Introduction	1
1.1 Outline and Summary of Contributions	3
2 Reservoir Computing	8
2.1 Training an RC	10
2.2 Forecasting with an RC	11
2.3 Echo State Networks	12
2.4 Constructing ESNs	15
2.5 Properties of Good Reservoirs	17
2.6 Evaluating Reservoirs	19
2.7 Summary	22
3 Low-Connectivity Reservoirs	23
3.1 Bayesian Optimization	24
3.2 Structure of Low-Connectivity Reservoirs	26
3.3 ESN Symmetries and their Consequences	28
3.4 Training, Forecasting, and Evaluation	30
3.5 Results	32
3.5.1 Forecasting Lorenz '63	32
3.5.2 ESN Variation	33
3.5.3 Reproducing the Lorenz '63 Attractor	35
3.5.4 Forecasting Rössler and the Double-Scroll Circuit	36
3.5.5 Cross-task Performance	36
3.6 Conclusion	37

4	RCs without Reservoirs: Nonlinear Vector Auto-regressions	40
4.1	Solutions to Linear ESNs	41
4.1.1	Prediction	41
4.1.2	Inference	42
4.2	Linear Vector Auto-regressions (VARs)	43
4.2.1	Training and Testing	44
4.2.2	Comparing VARs to ESNs	45
4.3	Nonlinear Vector Autoregressions (NVARs)	46
4.4	Practical Considerations	48
4.5	Summary	49
5	NVARs in Practice	51
5.1	State Inference with Lorenz '63	51
5.1.1	Output Weights	54
5.2	Forecasting Lorenz '63	54
5.2.1	z Return Map	57
5.2.2	Fixed Points	60
5.2.3	Output Weights	61
5.2.4	Computational Complexity	63
5.2.5	Robustness to Noise	64
5.3	Forecasting the Double-Scroll Circuit	64
5.3.1	Fixed Points	68
5.4	Forecasting Mackey-Glass	69
5.4.1	Fixed Points	72
5.4.2	Output Weights	72
5.5	Conclusion	74
6	Conclusion	76
	References	78
 Appendices		
A	Example Systems	84
A.1	Lorenz '63	84
A.2	Rössler	86
A.3	Double-Scroll	88
A.4	Mackey-Glass	88

List of Figures

Figure	Page
1.1 High-level view of an echo state network RC. An input signal (red, top left) drives an internal reservoir to produce a reservoir response (blue, top center) which is then combined to form an output signal (green, top right). In an ESN, the reservoir network has external input signals (red nodes, left) that drive the internal network (blue nodes, center) to produces an overall output (green, right). Each internal node may have two kinds of input connections: connections to other nodes in the network (W_r , blue arrows), or connections to the overall input (W_{in} , red arrows). Each node may also contribute to the overall output (W_{out} , green arrows). Note that the internal connections may contain cycles. When the ESN is used to perform forecasting, the output on the right side is connected to the input on the left side, allowing the ESN to run autonomously with no external input.	3
1.2 The five reservoir structures tested. Only internal reservoir connections are pictured. Connections to the reservoir computer input, or to the output layer are not shown. (a) A general, fixed in-degree network, here pictured with $N = 7$ and $k = 2$. (b) A $k = 1$ network with a single connected component. (c) A $k = 1$ network with the single cycle cut at an arbitrary point. (d) A <i>simple cycle reservoir</i> . (e) A <i>delay line reservoir</i>	5
1.3 Summary of the (N)VAR method. Many time-delay taps τ_i of the discrete-time signal $\mathbf{u}(t)$ (top, red) are concatenated into the tap vector $\mathbf{v}(t)$ (middle, blue). Here, there are three taps $\tau_0 = 0$, $\tau_1 = 2$, and $\tau_2 = 4$. These taps are then passed through a possibly nonlinear function $\mathbf{g}_n$ and combined linearly by the output matrix W_{out} to produce the next value of the (N)VAR's output $\mathbf{y}(t)$ (bottom, green). For a linear VAR, $\mathbf{g}_n$ is the identity function. To transform a whole time series input, this (N)VAR process slides along the time axis from left to right.	5
1.4 The true Lorenz attractor (a) and NVAR predicted attractor (e) for a single training trial. (b) – (d) True Lorenz system (blue) during training overlaid with NVAR output (red) calculated after training is complete. (f) – (h) True (blue) and NVAR forecasted output (red). The NVAR shows good agreement with the true system as far as 6 Lyapunov periods in to the autonomous forecast.	6

2.1	High-level view of an echo state network RC. An input signal (red, top left) drives an internal reservoir to produce a reservoir response (blue, top center) which is then combined to form an output signal (green, top right). In an ESN, the reservoir network has external input signals (red nodes, left) that drive the internal network (blue nodes, center) to produces an overall output (green, right). Each internal node may have two kinds of input connections: connections to other nodes in the network (W_r , blue arrows), or connections to the overall input (W_{in} , red arrows). Each node may also contribute to the overall output (W_{out} , green arrows). Note that the internal connections may contain cycles. When the ESN is used to perform forecasting, the output on the right side is connected to the input on the left side, allowing the ESN to run autonomously with no external input.	13
2.2	Summary of the forecasting evaluation method. I calculate errors $\epsilon_{1,i}$ at times t_i , marked on the attractor as red dots, and combine them into a single error $\tilde{\epsilon}$. Before each t_i (dotted vertical line), the reservoir is integrated with Eq. (2.1) and listening to the input. After t_i , the reservoir is integrated with Eq. (2.7), and runs autonomously. The reservoir's prediction (dotted red line) must eventually diverge from the true system (solid black line). I calculate each $\epsilon_{1,i}$ only during a single Lyapunov period after forecasting begins, marked here in gray.	21
3.1	The five reservoir structures tested. Only internal reservoir connections are pictured. Connections to the reservoir computer input, or to the output layer are not shown. (a) A general, fixed in-degree network, here pictured with $N = 7$ and $k = 2$. (b) A $k = 1$ network with a single connected component. (c) A $k = 1$ network with the single cycle cut at an arbitrary point. (d) A <i>simple cycle reservoir</i> . (e) A <i>delay line reservoir</i>	26
3.2	The forecasted attractor from an ESN with full inversion symmetry trained on the Lorenz '63 system, using identity read-out, in the x/z plane. Compare against Fig. A.2, and note the two deformed images of the butterfly attractor on top of each other, with one flipped along the z axis. Breaking this inversion symmetry in the read-out function is enough to prevent this kind of failure entirely.	29
3.3	Two examples of ESNs that fail to reproduce the Lorenz attractor, but produce a low ϵ_1 . Compare with the true attractor in Fig. A.2. Network (i) fails to learn the attractor, overemphasizing the right lobe in which t_1 resides. Network (ii) matches the attractor well early on, but as the prediction lengthens, it falls into a periodic orbit (thick black line). Both (i) and (ii) show a promising low ϵ_1 , but the averaged $\tilde{\epsilon}$ measure more accurately captures their failure to learn the Lorenz attractor.	32

3.4	Performance for each network structure, evaluated for the Lorenz system at the meta-parameters listed in Table 3.2. Each is visualized as a Gaussian kernel density estimation in $\log_{10} \tilde{\epsilon}$ with a bandwidth of 0.35 [36], which can be interpreted as a probability distribution. Using a narrower bandwidth does not reveal additional features. A vertical line marks the median. Note that structures (b) – (e) have very long tails, and can produce networks that perform very poorly in comparison to (a). However, all five have the capability to produce well-performing networks.	34
3.5	Lorenz attractor plots in the x/z plane for long-term free-running predictions for each network structure in Table 3.2. Compare with true Lorenz attractor in Fig. A.2, and the failed reservoirs in Fig. 3.3.	35
4.1	Summary of the (N)VAR method. Many time-delay taps τ_i of the discrete-time signal $\mathbf{u}(t)$ (top, red) are concatenated into the tap vector $\mathbf{v}(t)$ (middle, blue). Here, there are three taps $\tau_0 = 0$, $\tau_1 = 2$, and $\tau_2 = 4$. These taps are then passed through a possibly nonlinear function $\mathbf{g}_n$ and combined linearly by the output matrix W_{out} to produce the next value of the (N)VAR’s output $\mathbf{y}(t)$ (bottom, green). For a linear VAR, $\mathbf{g}_n$ is the identity function. To transform a whole time series input, this (N)VAR process slides along the time axis from left to right.	43
5.1	(a) – (c) The Lorenz ’63 system (blue) during the NVAR training. After the NVAR is trained, it is re-used on the training signal to produce the training output (c, red). (d) – (f) The Lorenz ’63 system during NVAR testing, with the NVAR output. In both cases, the NVAR output (red) lies directly on top of the true Lorenz signal (blue).	53
5.2	Weights in the matrix W_{out} (left) for the NVAR trained for Lorenz system forecasting. On the right, a magnified view of the same. The largest component is the constant offset, to account for the fact that the z component of the Lorenz system is positive, even when both x and y are zero. Also note that the linear terms in x and y (highlighted in red) have weights near zero. As the Lorenz system has a $(x, y) \rightarrow (-x, -y)$ symmetry, the NVAR must have zero weights attached to these terms to respect that symmetry.	55
5.3	The true Lorenz attractor (a) and NVAR predicted attractor (e) for a single training trial. (b) – (d) True Lorenz system (blue) during training overlaid with NVAR output (red) calculated after training is complete. (f) – (h) True (blue) and NVAR forecasted output (red). The NVAR shows good agreement with the true system as far as 6 Lyapunov periods in to the autonomous forecast.	56
5.4	(a) The z return map of the Lorenz ’63 system, integrated using various methods. (b) Detail of the region marked in (a). Using a relative integration tolerance of 10^{-3} , the Runge-Kutta 3(2) (blue +) differs from R-K 5(4) (orange $\times$) in both position and spread. Both methods converge after sharpening the tolerance to 10^{-5} (green square and red triangle, respectively).	58

5.5	(a) The z return map of the Lorenz '63 system (blue +) overlaid with the z return map of the NVAR forecast (red $\times$), trained for 10 time units. The NVAR reproduces the long-term dynamics of the z variable accurately enough at this scale that it is difficult to see the true return map underneath. (b) Detail of the region marked in (a). At this scale, it is possible to see that the NVAR return map is wider than the true Lorenz return map, although they lie on top of each other. This reproduction can be improved by training the NVAR for longer (see Fig. 5.6).	59
5.6	(a) The z return map of the Lorenz '63 system (blue +) overlaid with the z return map of the NVAR forecast (red $\times$), trained for 20 time units. (b) Detail of the region marked in (a). With more training time, the NVAR return map is narrower and more accurately reproduces the Lorenz return map.	59
5.7	Weights in the matrix W_{out} for the NVAR trained for Lorenz system forecasting. Terms that appear directly in the Lorenz system (Eq. (A.1)) are in dashed red. There does not appear to be any correlation between a large weight in the NVAR output, and presence in the Lorenz equations.	62
5.8	The long-term trajectory of a noise-driven Lorenz '63 system (a) and NVAR predicted attractor (e) for a single training trial. (b) – (d) Noise-driven Lorenz system (blue) during training overlaid with NVAR output (red) calculated after training is complete. (f) – (h) True, noise-free Lorenz (blue) and NVAR forecasted output (red).	65
5.9	The true double-scroll attractor (a) and NVAR predicted attractor (e) for a single training trial. (b) – (d) True double-scroll system (blue) during training overlaid with NVAR output (red) calculated after training is complete. (f) – (h) True (blue) and NVAR forecasted output (red). Here, the NVAR shows good agreement with the true system as far as 6 Lyapunov periods.	67
5.10	The true Mackey-Glass attractor (a) and NVAR predicted attractor (c) for a single training trial. (b) True Mackey-Glass system during training overlaid with NVAR output (red) calculated after training is complete. (d) True (blue) and NVAR forecasted output (red). Here, the NVAR shows good agreement with the true system as far as 2 Lyapunov periods.	71
5.11	Weights in the matrix W_{out} for the NVAR trained for Mackey-Glass system forecasting. Terms that break the inversion symmetry are in dashed red. In labels, u_i is shorthand for $u(t - \tau_i \Delta t)$. As the training signal does not reflect the inversion symmetry, the red terms are not trained to near zero as they are in Fig. 5.2.	73
A.1	The true Lorenz attractor, produced by integrating Eq. (A.1).	85
A.2	The true Lorenz attractor projected on to the x/z plane.	85
A.3	The true Rössler attractor, produced by integrating Eq. (A.2).	86
A.4	The true double-scroll circuit attractor, produced by integrating Eq. (A.3).	87
A.5	The true double-scroll circuit attractor projected on to the V_1/I plane.	87
A.6	The true Mackey-Glass attractor represented as a time delay embedding using τ as the delay, produced by integrating Eq. (A.4).	89

List of Tables

Table		Page
1.1	A summary of chapters and associated contributions.	4
2.1	A summary of variations of the ESN equation. The discrete-time formulation is the result of Euler-integrating the continuous-time formulation, and the forecasting variant is derived via the substitution described in Section 2.2. .	14
2.2	Summary of echo state network meta-parameters. These parameters must be chosen to produce a random realization of an ESN-based RC.	17
3.1	Range of meta-parameters searched using Bayesian optimization.	25
3.2	Best reservoir computers of each reservoir structure, after 100 iterations of the Bayesian optimization algorithm using the Lorenz system as input. The meta-parameters chosen by the algorithm are shown on the right. The simpler structures (b) – (e) all perform nearly as well as the general structure (a). .	33
3.3	Result of optimizing reservoirs with the Bayesian algorithm over 100 iterations, on both the Rössler system and the double-scroll system. All five structures can perform equally well at both systems.	36
3.4	Result of re-using reservoirs optimized for Lorenz prediction to perform double-scroll prediction. The minimum error encountered across all 20 reservoirs of each topology is reported as $\tilde{\epsilon}_{\min}$	37
4.1	Summary of NVAR parameters. Compared to the ESN approach in Table 2.2, the NVAR method is much simpler.	48
5.1	Estimated speed-up of the NVAR on the Lorenz '63 forecasting task, compared to ESNs described in Chapter 3 [18] and in other work [11, 23]. . . .	64
5.2	Summary of results in this chapter: the dimension of W_{out} , the ridge parameter α , the NRMSE (ϵ for inference, $\tilde{\epsilon}$ for forecasting), and the normalized distance from true fixed points to predicted fixed points.	74

Chapter 1

INTRODUCTION

Recently, many difficult problems have been solved using methods from the field of machine learning, in particular using neural networks. Machine learning methods work by learning solutions from example data alone, without the need for an existing model for that data. Physicists have used neural networks to find the masses of particles from decay data [1] and separate quark and gluon jets in e^+e^- annihilation [2]. In other fields, this approach has been used effectively on handwritten digit recognition [3, 4], speech recognition [5], and speech production [6]. These are all problems with large sets of example data, but which are otherwise difficult or intractable with a more traditional programming approach. Put simply, it is easy to provide examples of what the number “2” looks like, while it is difficult to define what features a written “2” must have in a programming language.

By its nature, many problems in physics are defined in terms of time-varying data. Neural networks that contain internal cycles, called recurrent neural networks, are a natural fit for these problems. The addition of cycles gives the resulting network memory and allows it to be used to effectively solve many time-domain problems, such as forecasting chaotic systems [7]. However, this modification has a cost: recurrent neural networks take much more time to train, as the training algorithm takes exponentially longer to learn how to utilize past inputs [8, 9].

More recently, a new machine learning tool has emerged for dealing with time-domain problems. Known as *reservoir computing*, this tool was originally formulated as a novel way to reduce the computational cost of training recurrent neural networks [9]. Reservoir

computers (RCs) have been effectively applied to chaotic system forecasting [10, 11] and system state inference [12]. More than just short-term prediction, RCs are capable of reproducing the *climate* of a dynamical system [11, 13], that is, it learns the long-term features of the system such as the system’s attractor and Lyapunov exponents.

RCs are interesting to physicists not only as a tool to solve problems. At the heart of a RC is a dynamic system called the *reservoir*, which provides the rich dynamics the RC draws on to function. This is usually implemented as a neural network, but in practice many dynamic systems function well. This opens the door to building RCs around real physical systems, such as a network of autonomous logic on an FPGA [14] or an optical feedback system [15]. This is interesting from a theoretical perspective, as physical RCs use an ancillary system to help answer questions about a main system, but they are also practically interesting: by relying on physical reservoirs, rather than simulation, these methods perform chaotic system forecasting and other tasks at a very high rate.

As a new field, reservoir computing still has many unanswered fundamental questions. What qualities should the internal reservoir have to create a good RC? A common practice is to use a random network as the reservoir, generated from a set of meta-parameters. There are a handful of heuristics for some parameters, such as setting the *spectral radius* of the network near one and the need for recurrent network connections [16, 17], but the applicability of these rules is narrow. What features of the reservoir are necessary, and which can be discarded?

In this dissertation, I construct reservoir computers that do not follow the conventional rules for reservoir construction. These RCs continue to perform well on chaotic system forecasting and state inference, and can even reproduce the chaotic attractor of the system they have learned, despite the fact that their internal reservoir has no recurrent connections. Indeed, by the end of this dissertation, they do not have an internal reservoir at all.

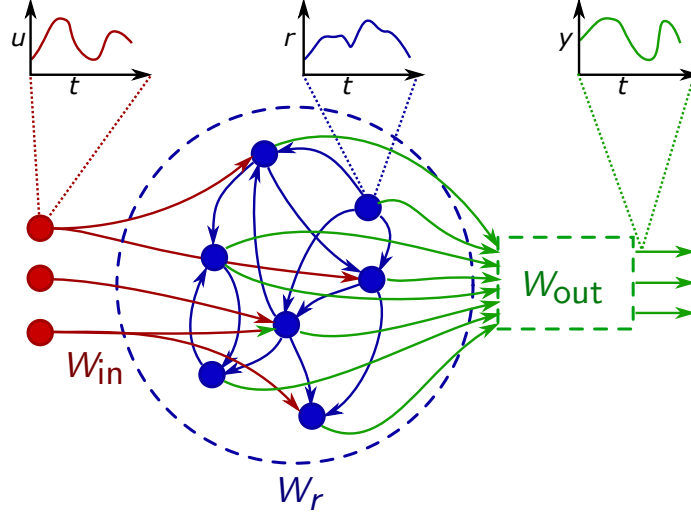


Figure 1.1: High-level view of an echo state network RC. An input signal (red, top left) drives an internal reservoir to produce a reservoir response (blue, top center) which is then combined to form an output signal (green, top right). In an ESN, the reservoir network has external input signals (red nodes, left) that drive the internal network (blue nodes, center) to produces an overall output (green, right). Each internal node may have two kinds of input connections: connections to other nodes in the network (W_r , blue arrows), or connections to the overall input (W_{in} , red arrows). Each node may also contribute to the overall output (W_{out} , green arrows). Note that the internal connections may contain cycles. When the ESN is used to perform forecasting, the output on the right side is connected to the input on the left side, allowing the ESN to run autonomously with no external input.

1.1 Outline and Summary of Contributions

In this dissertation, Chapters 2 and 4 provide background information on the methods used, while Chapters 3 and 5 present new contributions. These contributions are summarized in Table 1.1.

In Chapter 2, I describe the reservoir computing method, as well as a common concrete implementation of a RC called an *echo state network* (ESN) summarized in Fig. 1.1. I explain how to construct an ESN, the meta-parameters that govern its construction, and discuss some of the limited known results for choosing these meta-parameters. Then, I discuss how the RC is trained, both for autonomous forecasting tasks and for inference tasks, as well as how performance quality on these tasks is commonly measured. I briefly discuss the known problems with this performance measure, and suggest a new measure

	Chapter	Contribution
	Chapter 2 Reservoir Computing	• introduce a new metric $\tilde{\epsilon}$
[18]	Chapter 3 Low-Connectivity Reservoirs	• demonstrate new metric $\tilde{\epsilon}$ • explore ESNs with very low connectivity • demonstrate these ESNs perform equally well as traditional ESNs
[19]	Chapter 5 NVARs in Practice	• demonstrate NVARs on forecasting/inference • show that NVARs perform as well as ESNs even with a low number of taps • explore current limitations of NVAR approach

Table 1.1: A summary of chapters and associated contributions.

designed to mitigate these problems.

In Chapter 3, I apply ESNs to the task of chaotic system forecasting for three different chaotic systems, using my new metric. I demonstrate that this new metric more accurately reflects some long-term forecasting failures that the traditional metric does not. I use a Bayesian optimization algorithm to tune the meta-parameters for the ESNs, and in so doing, discover a class of well-performing ESNs with very simple internal reservoirs. I identify four classes of reservoir networks with particularly simple structure, summarized in Fig. 1.2. Of these, two had been previously identified. I build on this work and demonstrate that all four classes have the capacity to perform equally well as a traditional ESN. In particular, all four classes can reproduce the chaotic attractor of the system they are trained on. I then discuss the trade-offs between these simpler reservoir structures and a traditional ESN.

In Chapter 4, I describe a different technique known as nonlinear vector auto-regression (NVAR), which has recently been shown to be mathematically equivalent to an ESN under certain conditions. This technique is summarized in Fig. 1.3. I outline this proof, discuss the construction and training of an NVAR, and describe how to apply it to common RC tasks. Then, I discuss the parallels between the NVAR and RC approaches, as well as the practical considerations of applying this equivalence proof. Finally, I discuss the trade-offs between the two approaches.

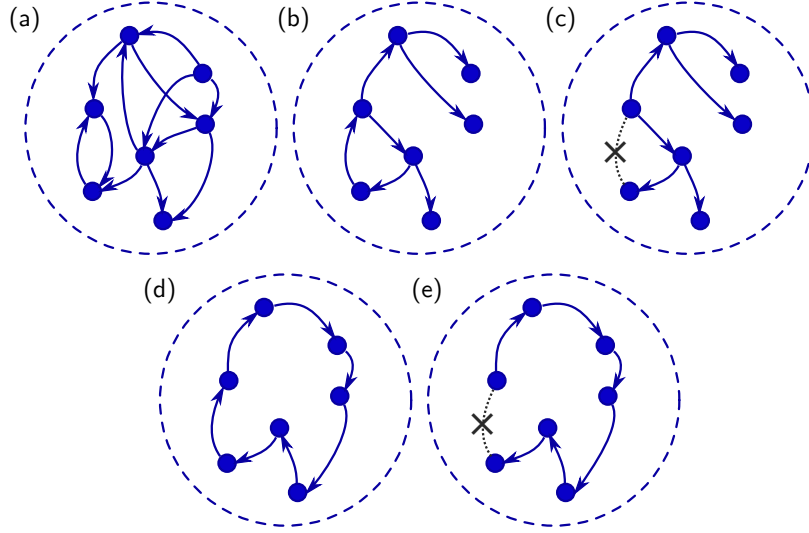


Figure 1.2: The five reservoir structures tested. Only internal reservoir connections are pictured. Connections to the reservoir computer input, or to the output layer are not shown. (a) A general, fixed in-degree network, here pictured with $N = 7$ and $k = 2$. (b) A $k = 1$ network with a single connected component. (c) A $k = 1$ network with the single cycle cut at an arbitrary point. (d) A *simple cycle reservoir*. (e) A *delay line reservoir*.

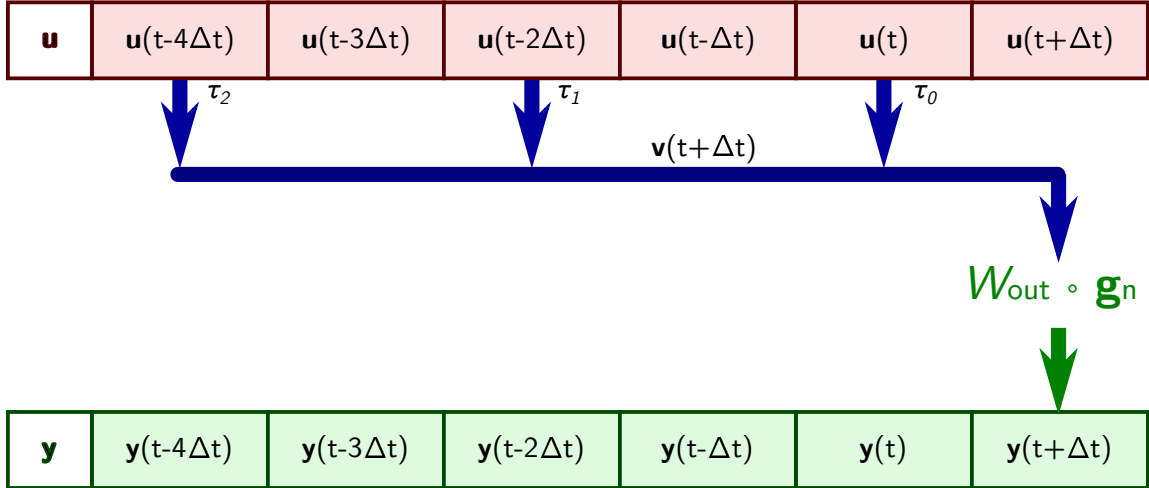


Figure 1.3: Summary of the (N)VAR method. Many time-delay taps τ_i of the discrete-time signal $\mathbf{u}(t)$ (top, red) are concatenated into the tap vector $\mathbf{v}(t)$ (middle, blue). Here, there are three taps $\tau_0 = 0$, $\tau_1 = 2$, and $\tau_2 = 4$. These taps are then passed through a possibly nonlinear function $\mathbf{g}_n$ and combined linearly by the output matrix W_{out} to produce the next value of the (N)VAR's output $\mathbf{y}(t)$ (bottom, green). For a linear VAR, $\mathbf{g}_n$ is the identity function. To transform a whole time series input, this (N)VAR process slides along the time axis from left to right.

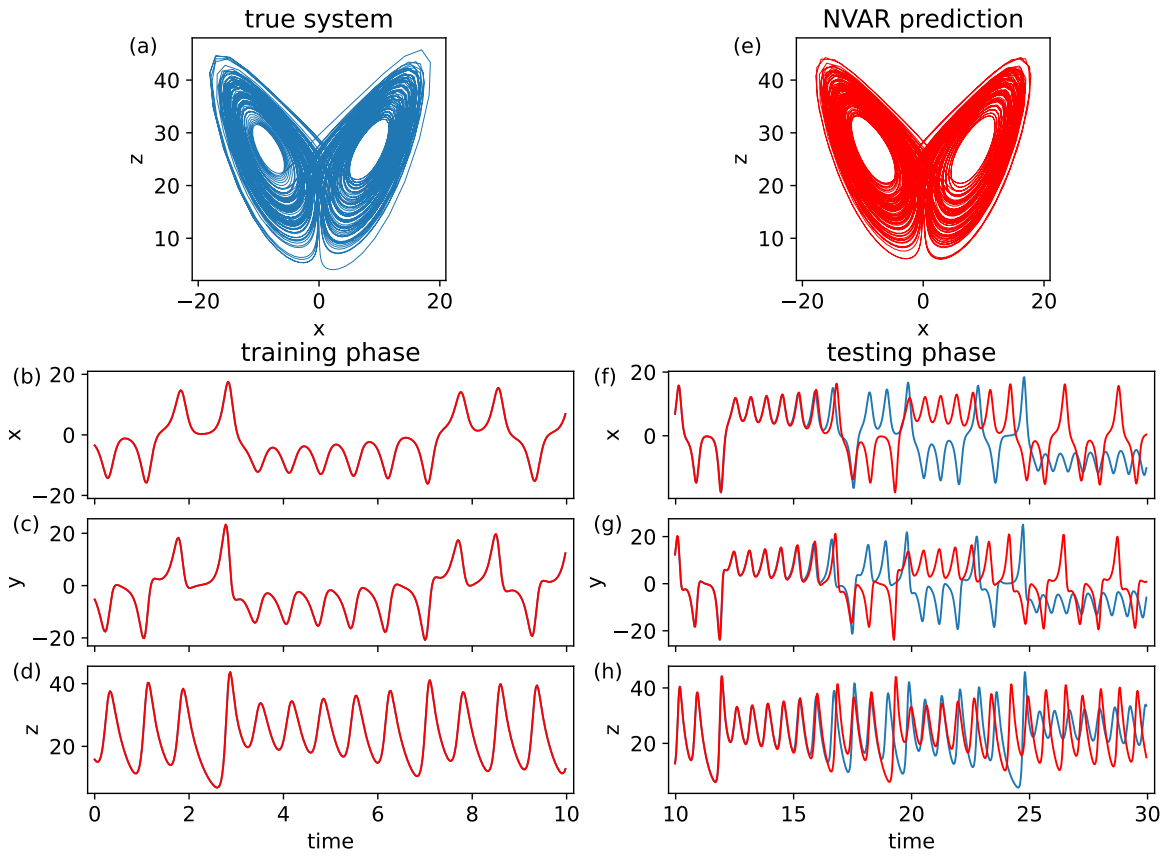


Figure 1.4: The true Lorenz attractor (a) and NVAR predicted attractor (e) for a single training trial. (b) – (d) True Lorenz system (blue) during training overlaid with NVAR output (red) calculated after training is complete. (f) – (h) True (blue) and NVAR forecasted output (red). The NVAR shows good agreement with the true system as far as 6 Lyapunov periods in to the autonomous forecast.

In Chapter 5, I build on this equivalence with my collaborators to demonstrate NVARs solving two common RC tasks: system forecasting (as in Fig. 1.4) and state inference. I show that NVARs are capable of ESN-equivalent performance on these tasks even under conditions where the equivalence proof does not strictly hold. The NVARs achieve this performance despite being considerably simpler than an ESN and with very little training data and training time. In addition, I discuss how a trained NVAR can provide interpretable results, while a trained RC is opaque. I then show how the NVAR approach can be extended naturally to solve a wider array of problems. Finally, I discuss how this approach may replace traditional RCs, and what questions remain to be answered before that can happen.

Finally, in Chapter 6, I summarize my findings and contributions, and propose interesting new directions for future research discovered during this work.

Chapter 2

RESERVOIR COMPUTING

At a high level, a *reservoir computer* (RC) is a method to transform one time-varying signal $\mathbf{u}(t)$, the input to the RC, into another time-varying signal $\mathbf{y}(t)$, the output of the RC. The RC is constructed in such a way that, given an example input and a desired output, the RC can be *trained* to produce that output when given the corresponding input. Once trained, the RC can then be used to perform the trained transformation on inputs it has not seen before.

The RC does this by means of an internal dynamic system called the *reservoir*, which is coupled to the input vector $\mathbf{u}(t)$. If the reservoir's dynamics can be expressed as a first-order differential equation, which covers a broad range of interesting choices of reservoir, then the reservoir's internal state vector $\mathbf{r}$ is given by

$$\dot{\mathbf{r}}(t) = \mathbf{R}(\mathbf{r}, \mathbf{u}, t), \quad (2.1)$$

where the vector function $\mathbf{R}$ defines the dynamics of the reservoir. For most systems, $\mathbf{R}$ is a function of time only implicitly through $\mathbf{r}(t)$ and $\mathbf{u}(t)$.

The reservoir is commonly a specific type of system called an *echo state network*, discussed in Section 2.3, but this is not at all a requirement. Effective RCs have been built around autonomous Boolean networks [14], optical feedback systems [15], single time-delay Boolean nodes [20], and many others. The specific properties that a reservoir must have in order to produce a functioning RC is an open question. Though there are a few known results, which I will discuss in Section 2.5, the most direct and practical way to know if a

given reservoir will work is to build, train, and test it.

The output vector $\mathbf{y}(t)$ of the RC is a linear combination of a set of *read-out* signals, constructed from the reservoir state $\mathbf{r}(t)$ by means of a vector read-out function $\mathbf{g}$ and matrix W_{out} ,

$$\mathbf{y}(t) = W_{\text{out}} \mathbf{g}(\mathbf{r}(t)). \quad (2.2)$$

This is called the RCs *output layer*. The read-out function $\mathbf{g}$ selects which components of $\mathbf{r}(t)$ are used by the matrix W_{out} , possibly modifying them in the process. Often, $\mathbf{g}$ is the identity function, and the output signal $\mathbf{y}(t)$ is a direct linear combination of the state variables of the reservoir. However, a non-identity $\mathbf{g}$ can be used to break unwanted symmetries in the $\mathbf{u}$ -to- $\mathbf{y}$ transformation (as in Section 3.3), to introduce nonlinearity to an otherwise completely linear RC (as in Section 4.3), or to model incomplete measurements of a physical system being used as a reservoir.

Together, Eq. (2.1) and Eq. (2.2) define a reservoir computer, and define the transformation from input signal $\mathbf{u}(t)$ to output $\mathbf{y}(t)$.

Intuitively, the input signal $\mathbf{u}(t)$ drives the reservoir, producing a large number of reservoir state signals $\mathbf{r}(t)$. These state signals may be quite complicated, and likely none of them match the desired transformation, but they are combined by means of W_{out} to produce the desired transformation. The reservoir's purpose is to broadcast the input signal into a high-dimensional space, and the output layer's purpose is to combine those many dimensions into the meaningful desired output of the RC.

The reservoir dynamics $\mathbf{R}$ and the readout function $\mathbf{g}$ are considered a fixed part of the RCs construction, and are not part of the training process. I will discuss this choice in more detail in Sections 2.3 and 2.5. Once these design parameters are fixed, the weight matrix W_{out} can be calculated from an example input and an example output by a process called *training*, which results in a complete RC that performs the desired transformation.

2.1 Training an RC

Training an RC starts with an example input signal $\mathbf{u}_{\text{train}}(t)$ and corresponding example output signal $\mathbf{y}_{\text{train}}(t)$. As an example, $\mathbf{u}_{\text{train}}(t)$ might be the x and y components of a three-dimensional dynamic system, and $\mathbf{y}_{\text{train}}(t)$ would be the z component. An RC trained on this example would learn how to infer the z component of the system from the other two.

The first step is to feed the example input $\mathbf{u}_{\text{train}}(t)$ into the reservoir, producing an example reservoir response $\mathbf{r}_{\text{train}}(t)$. Using Eq. (2.2), the final goal of training is to find a matrix W_{out} such that

$$\mathbf{y}_{\text{train}}(t) \approx W_{\text{out}} \mathbf{g}(\mathbf{r}_{\text{train}}(t)). \quad (2.3)$$

For many practical reservoirs, it takes some time for the reservoir state $\mathbf{r}(t)$ to synchronize with the input $\mathbf{u}(t)$. During this time, the reservoir state still depends on its initial condition, and so is unsuitable for use as training data. Practically, if the example data starts at $t = 0$, then all the data before $t < t_{\text{warmup}}$ is discarded, and the approximation in Eq. (2.3) need only be true for $t > t_{\text{warmup}}$. The warm-up time t_{warmup} will depend on the specific choice of reservoir in the RC.

The right-hand side of Eq. (2.3) is linear in W_{out} , and can be solved by any number of linear regression methods. For RCs, W_{out} is most commonly found via ridge regression, also known as Tikhonov regularization. This chooses W_{out} to minimize

$$\alpha ||W_{\text{out}}||^2 + \sum_{t=t_{\text{warmup}}}^{t_{\text{train}}} |\mathbf{u}_{\text{train}}(t) - W_{\text{out}} \mathbf{r}_{\text{train}}(t)|^2. \quad (2.4)$$

where t_{train} is the end time of the example input $\mathbf{u}_{\text{train}}(t)$. In most practical applications, the signals involved are either measured at a fixed time step, or integrated at a fixed time step, and the sum in Eq. (2.4) is understood to be over these fixed time steps.

The ridge parameter α is included to prevent over-fitting, and to help the RC generalize from the example input to unknown inputs. In practice, this value depends on the scale of the input signal $\mathbf{u}_{\text{train}}(t)$; for roughly unit-scale inputs, α may be anywhere from 10^{-9} to 10^9 . Since the ridge regression can be calculated quickly, and the result is only logarithmically

sensitive to the value of α , the best value for α can be found simply by grid search or cross-validation on the example input.

The fact that training an RC amounts to a linear regression is a major feature worth highlighting. Linear regressions are very fast, compared to other machine learning techniques like gradient descent and backpropagation [9]. As a result, training an RC is also fast. This opens the door to many interesting applications where the ability to rapidly retrain the RC to a new system without the aid of a supercomputer is desirable. For example, an RC that controls the motors on an airborne drone could retrain itself in the event of a mechanical failure.

2.2 Forecasting with an RC

One of the primary uses of an RC is *system forecasting*, where the RC is primed with an input signal from a dynamic system and then switched into a free-running mode, where it produces a future forecast of that signal. This is most commonly used to forecast chaotic systems, where forecasting is definitionally difficult.

Training an RC for forecasting starts with an example input signal $\mathbf{u}_{\text{train}}(t)$, but does not need an example output $\mathbf{y}_{\text{train}}(t)$ as in normal training. As before, feeding the example input into the reservoir generates an example reservoir response $\mathbf{r}_{\text{train}}(t)$. For forecasting, however, the matrix W_{out} is chosen so that

$$\mathbf{u}_{\text{train}}(t) \approx W_{\text{out}} \mathbf{g}(\mathbf{r}_{\text{train}}(t)). \quad (2.5)$$

That is, the RC is trained to reproduce its input as its output,

$$\mathbf{u}(t) \approx \mathbf{y}(t). \quad (2.6)$$

As before, W_{out} is found via ridge regression.

Once trained in this way, the RC can be used for forecasting. First, the RC is primed with an input signal $\mathbf{u}(t)$ to be used as the basis for forecasting. This priming step synchronizes the reservoir state $\mathbf{r}(t)$ to the input, and intuitively provides the RC with the past history

it will draw on to produce the forecast. This is a warm-up step, similar to that used during training. At the moment forecasting begins, the approximation Eq. (2.5) is substituted in to the reservoir equation Eq. (2.1), effectively connecting the input of the RC to the output. The RC now evolves forward in time according to

$$\dot{\mathbf{r}}(t) = \mathbf{R}(\mathbf{r}, W_{\text{out}} \mathbf{g}(\mathbf{r}), t), \quad (2.7)$$

Note that this RC no longer has any external input and runs autonomously. The RC output $\mathbf{y}(t)$, calculated as usual from the output layer in Eq. (2.2), is the forecasted signal.

2.3 Echo State Networks

Although there are many choices for the dynamic reservoir at the heart of an RC, the reservoir computing method originated as a practical way to train recurrent neural networks by only training the weights in the output layer [9]. One such network reservoir design, known as an *echo state network* (ESN) [16], remains popular and will be my choice of dynamic reservoir for this dissertation.

An echo state network is composed of number of nodes N . Each node in the network has a number of inputs, drawn either from other nodes in the network or from the external driving system. Each node also has an internal state, a single value that decays exponentially to zero over time with a decay rate γ . The node's inputs are summed together with weights, and this sum is passed through an activation function f to produce a driving signal to the node's exponential state, which is also used as the node's output.

The input connections and weights for each node are represented by two matrices: W_{in} , for connections from the overall input $\mathbf{u}(t)$, and W_r , for recurrent connections to other nodes inside the network. For example, $W_{r,i,j}$ represents a connection from node j inside the network to node i , while $W_{\text{in},i,j}$ represents a connection from input component $u_j(t)$ to node i . This means that W_r is a $N \times N$ matrix, while W_{in} is an $N \times d$, where d is the dimension of the input signal $\mathbf{u}(t)$.

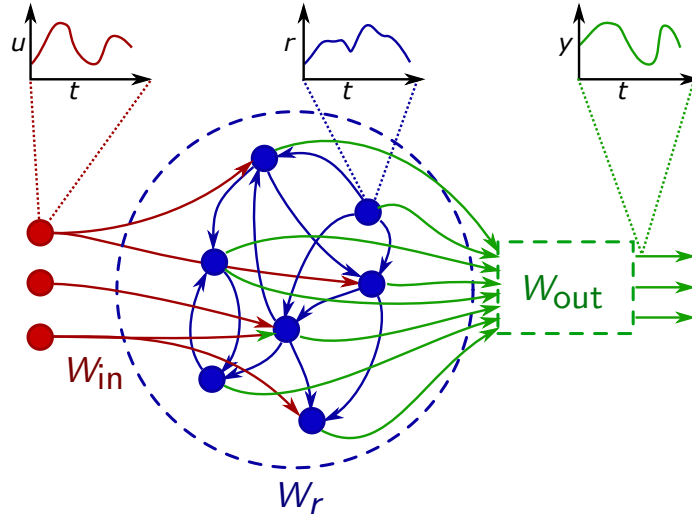


Figure 2.1: High-level view of an echo state network RC. An input signal (red, top left) drives an internal reservoir to produce a reservoir response (blue, top center) which is then combined to form an output signal (green, top right). In an ESN, the reservoir network has external input signals (red nodes, left) that drive the internal network (blue nodes, center) to produces an overall output (green, right). Each internal node may have two kinds of input connections: connections to other nodes in the network (W_r , blue arrows), or connections to the overall input (W_{in} , red arrows). Each node may also contribute to the overall output (W_{out} , green arrows). Note that the internal connections may contain cycles. When the ESN is used to perform forecasting, the output on the right side is connected to the input on the left side, allowing the ESN to run autonomously with no external input.

Variation		Equations
(a)	Continuous (inference)	$\dot{\mathbf{r}}(t) = -\gamma \mathbf{r}(t) + \gamma \mathbf{f}(W_r \mathbf{r}(t) + W_{\text{in}} \mathbf{u}(t))$ $\mathbf{y}(t) = W_{\text{out}} \mathbf{g}(\mathbf{r}(t))$
(b)	Continuous (forecasting)	$\dot{\mathbf{r}}(t) = -\gamma \mathbf{r}(t) + \gamma \mathbf{f}(W_r \mathbf{r}(t) + W_{\text{in}} W_{\text{out}} \mathbf{g}(\mathbf{r}(t)))$ $\mathbf{y}(t) = W_{\text{out}} \mathbf{g}(\mathbf{r}(t))$
(c)	Discrete (inference)	$\mathbf{r}(t + \Delta t) = (1 - \gamma \Delta t) \mathbf{r}(t) + \gamma \Delta t \mathbf{f}(W_r \mathbf{r}(t) + W_{\text{in}} \mathbf{u}(t))$ $\mathbf{y}(t + \Delta t) = W_{\text{out}} \mathbf{g}(\mathbf{r}(t + \Delta t))$
(d)	Discrete (forecasting)	$\mathbf{r}(t + \Delta t) = (1 - \gamma \Delta t) \mathbf{r}(t) + \gamma \Delta t \mathbf{f}(W_r \mathbf{r}(t) + W_{\text{in}} W_{\text{out}} \mathbf{g}(\mathbf{r}(t)))$ $\mathbf{y}(t + \Delta t) = W_{\text{out}} \mathbf{g}(\mathbf{r}(t + \Delta t))$

Table 2.1: A summary of variations of the ESN equation. The discrete-time formulation is the result of Euler-integrating the continuous-time formulation, and the forecasting variant is derived via the substitution described in Section 2.2.

Put together, the dynamics of an echo state network RC are

$$\begin{aligned}
\dot{\mathbf{r}}(t) &= -\gamma \mathbf{r}(t) + \gamma \mathbf{f}(W_r \mathbf{r}(t) + W_{\text{in}} \mathbf{u}(t)), \\
\mathbf{y}(t) &= W_{\text{out}} \mathbf{g}(\mathbf{r}(t)),
\end{aligned} \tag{2.8}$$

where the vector function $\mathbf{f}$ is understood to mean applying the one-dimensional activation function f component-by-component,

$$f_i(\mathbf{x}) = f(x_i). \tag{2.9}$$

This construction is summarized in Fig. 2.1. I will discuss choosing the matrices W_{in} and W_r , the parameter γ , and the activation function f in Section 2.4.

This is the continuous-time formulation of an ESN. It is also common to see a discrete-time formulation, which can be derived simply from Eq. (2.8) by the application of forward Euler integration. Additionally, the forecasting method described in Section 2.2 involves a modification to the RC equation once the RC is trained. These variations on the ESN equation are summarized in Table 2.1.

The Euler integration is only an accurate representation of Eq. (2.8) when $\gamma \Delta t \ll 1$. However, many authors will take $\gamma \Delta t = 1$. Although this is no longer an accurate approximation to the continuous-time ESN in Eq. (2.8), these discrete-time ESNs still function well

as an RC. That the RC is not sensitive to this kind of fundamental change in the underlying reservoir is a hallmark of the RC method. The specific form of the underlying reservoir is largely unimportant, except for a few requirements to be discussed in Section 2.5. Indeed, the internal structure of an ESN is determined randomly.

2.4 Constructing ESNs

The parameters W_r , W_{in} , γ , and f in Eq. (2.8) must be set before the ESN construction is complete. Of these, γ has the most simple interpretation: it controls the exponential decay of the value of each node in the network. As the overall input to the ESN only influences the network via node inputs, if this decay rate is too slow then the network will not be able to react fast enough to its input to be useful for computing the output. If instead this rate is too high, it is possible the network will respond too quickly and have no memory of past inputs, which may be necessary for the ESN’s task. The rate γ only controls the dynamics of an individual node, however, and both of these effects can be mitigated by the overall dynamics of the entire network of nodes. For example, a network with very fast nodes may still retain useful memory of past inputs if it contains long recurrent loops. Network structure can also exacerbate a poor choice of γ , as a network made of fast nodes that also contains very short recurrent loops may begin to oscillate rapidly, which is undesirable if the ESN’s desired output does not contain such oscillation. In practice, this decay rate should be matched to the characteristic timescale of the output signal, and then can be manually tuned from there if needed.

If the read-out function $\mathbf{g}$ is taken to be the identity function, as is common, and the activation function f is linear, then the forecasting-mode equations in Table 2.1 are completely linear and have simple, oscillatory analytic solutions. These are known as *linear ESNs*. Linear ESNs are mathematically easy to work with, and as a result many of the known proofs about ESNs are in the context of linear ESNs. One of the most recent and prominent results is discussed further in Chapter 4.

However, because the solutions to a linear ESN in prediction mode are so simple, it

is not possible for them to reproduce the strange attractors of chaotic systems. To do this, the activation function f is used to introduce nonlinearity into the equations. The most common choice is the hyperbolic tangent, $f(x) = \tanh(x)$, and I will use this choice in this dissertation. This satisfies the nonlinearity requirement, and also ensures that each component of $\mathbf{r}(t)$ is bounded in $(-1, 1)$. Bounds on $\mathbf{r}(t)$ ensure that Eq. (2.8) is particularly well-behaved for numerical integrators. These bounds can also be used to put bounds on the output $\mathbf{y}(t)$, which may be useful if that output is driving a real-world system.

It is also possible to leave f linear, and introduce nonlinearity into the system with $\mathbf{g}$. These *output-layer nonlinear ESNs* are also discussed further in Chapter 4.

The matrices W_r and W_{in} , and therefore the topology and weights of the ESN, are chosen randomly. There are many random schemes in use, from Erdős-Rényi networks and small-world networks [13] to simpler cycle and line networks [21]. The scheme I use is simple. Although this dissertation only discusses RCs simulated in software, this scheme is designed with an eye towards hardware implementation, where the number of inputs to each node, and the total number of connections in the network, have an associated hardware cost.

For the internal connections W_r , I generate a random network where every node has a fixed number of inputs, or *in-degree*, k . For each node, I select k nodes in the network without replacement and use those as inputs to the current node. Each input is assigned a random weight drawn from a normal distribution with mean 0 and variance 1. This results in a connection matrix W'_r where each row has exactly k non-zero entries. Finally, I rescale the whole matrix by

$$W_r = \frac{\rho_r}{\text{SR}(W'_r)} W'_r, \quad (2.10)$$

where $\text{SR}(W'_r)$ is the spectral radius, or maximum absolute eigenvalue, of the matrix W'_r . This scaling ensures that $\text{SR}(W_r) = \rho_r$. This parameter is used to tune the overall scale of the recurrent weights, and therefore the importance of recurrent connections inside the network.

For W_{in} , I randomly connect each node to each RC input in $\mathbf{u}(t)$ with probability σ . The weight for each connection is drawn randomly from a normal distribution with mean 0

	Parameter	Description
N	Number of Nodes	the total number of nodes in the network.
γ	Leak Rate	controls the exponential decay of each node.
σ	Input Connectivity	the probability that any given input is connected to any given node.
ρ_{in}	Input Scale	the variance of the normally-distributed input connection weights.
k	Recurrent Connectivity	the number of input connections each node draws from other nodes in the network.
ρ_r	Recurrent Scale	the spectral radius of W_r . This controls the scale of recurrent connection weights.
f	Activation Function	the function computed at each node. This is usually $\tanh$.
g	Read-out Function	the output layer read-out. This is usually the identity function.
α	Ridge Parameter	regularization parameter for ridge regression

Table 2.2: Summary of echo state network meta-parameters. These parameters must be chosen to produce a random realization of an ESN-based RC.

and variance ρ_{in}^2 .

Together, σ and ρ_{in} are enough to generate a random instantiation of W_{in} , and k and ρ_r are enough to generate a random instantiation of W_r . The relative scales ρ_r and ρ_{in} determine the importance of the recurrent and external inputs, respectively. The entire ESN construction is then reduced to choosing these *meta-parameters*, summarized in Table 2.2.

2.5 Properties of Good Reservoirs

Which dynamic systems make for good reservoirs, and what properties are desired in a reservoir, remains an open question. There are a handful of known desirable properties that are used to guide reservoir design, and in particular ESN design. Chief among these is the *fading memory property*, also known as the echo state property, which states that the reservoir’s response to its input should have a vanishing dependence on the value of that

input signal in the distant past [16, 22].

More concretely, let $\mathbf{u}_1(t)$ and $\mathbf{u}_2(t)$ be two different RC input signals which may differ for $t < 0$ but are otherwise identical for $t > 0$, and let $\mathbf{r}_1(t)$ and $\mathbf{r}_2(t)$ be the reservoir's response to these inputs. The reservoir has fading memory if and only if $\mathbf{r}_1(t) \rightarrow \mathbf{r}_2(t)$ as $t \rightarrow \infty$.

This has many practical benefits for an RC. A reservoir with fading memory guarantees a consistent output for a given input, regardless of the path through state space the input takes in the distant past. This is particularly important as running an RC often involves resetting the internal reservoir, either in simulation or in hardware. The fading memory property ensures that the specific initial state of the reservoir does not matter, as long as it has a sufficiently long warm-up time as discussed in Section 2.1.

One early result for ESNs states that this fading memory property cannot hold in an ESN when the input $\mathbf{u}(t) = 0$ and the spectral radius of the matrix W_r is greater than one [16], as any two distinct initial reservoir states $\mathbf{r}_1(0)$ and $\mathbf{r}_2(0)$ must diverge exponentially as $t \rightarrow \infty$.

Likewise, an ESN with $\text{SR}(W_r) \ll 1$ is contracting, and will return to $\mathbf{r} = 0$ rapidly with zero input. These two considerations have combined in literature into a recommendation that the spectral radius of W_r be near, but not above, one [17]. However, both of these results only hold for zero input, which is never the case when using an ESN in practice. Once the ESN and the input system are considered as a single coupled system, the ESN can display fading memory even when $\text{SR}(W_r) > 1$, and may never return to $\mathbf{r} = 0$ even when $\text{SR}(W_r) \ll 1$. More recent work presented in Chapter 3 and elsewhere [11, 21] show that these spectral radius recommendations are not hard and fast rules. Nonetheless, this history has cemented spectral radius as the standard measure of scale for W_r in ESNs.

Another desirable property of the reservoir is *continuity of response* to the input: similar inputs to the reservoir should produce similar responses. This is especially important in real-world applications, where the input to the RC is likely to be noisy. This excludes a wide class of chaotic systems from consideration as a reservoir, as even very small differences in two input signals would be amplified. Likewise, a good reservoir should show *separability*

of inputs: it should produce different responses to different inputs. In the extreme case, a reservoir that produces a constant zero regardless of input would form a very poor RC, despite meeting the other criteria listed so far.

More recently, these requirements have been unified under the umbrella of *invertible generalized synchronization* (IGS) [23–25], which claims that a good reservoir is one that is synchronized to its input. Specifically, the reservoir $\mathbf{r}(t)$ is in a state of generalized synchronization with its input $\mathbf{u}(t)$ if there is a function $\mathbf{H}$ such that, as $t \rightarrow \infty$, $\mathbf{r}(t) \rightarrow \mathbf{H}(\mathbf{u}(t))$. This already implies the fading memory property. If $\mathbf{H}$ is continuous, this implies continuity of response, and if $\mathbf{H}$ is invertible, this implies separability. An invertible $\mathbf{H}$ also ensures that

$$\mathbf{u}(t) \approx \mathbf{H}^{-1}(\mathbf{r}(t)). \quad (2.11)$$

This is close to the form of the RC’s output during forecasting in Eq. (2.5), although there is no guarantee that $\mathbf{H}^{-1}$ is linear.

The IGS approach differs from the earlier design principles in that it considers the unified reservoir/input system together, rather than looking for desirable properties of the reservoir independently. Unfortunately, it yields very little actionable advice about building a reservoir to use, and testing for the presence of IGS in an RC computationally expensive. The quickest way to see if a reservoir will work is still to build an RC with it, train it, and test it. Nonetheless, this new approach unifies many older principles in the field and is one of the most interesting avenues for future research.

2.6 Evaluating Reservoirs

To evaluate the performance of a trained RC, it is common to use an example testing input/output signal pair, $\mathbf{u}_{\text{test}}(t)$ and $\mathbf{y}_{\text{test}}(t)$, that differs from the training signal. The input $\mathbf{u}_{\text{test}}(t)$ is fed in to the trained RC, producing the RC output $\mathbf{y}(t)$, which is then compared to the desired output $\mathbf{y}_{\text{test}}(t)$. Note that the same concerns with reservoir warm-up, as discussed in Section 2.1, apply here: the early part of the input test signal is used only to synchronize the reservoir, and is not used in the comparison. Often, the training

and testing signals are combined such that $\mathbf{u}_{\text{test}}$ follows directly after $\mathbf{u}_{\text{train}}$, which allows the training signal to function as the warm-up signal for the reservoir during the testing phase.

The most common metric for comparing the true output $\mathbf{y}(t)$ to the desired output $\mathbf{y}_{\text{test}}(t)$ is to calculate the normalized root-mean-square error (NRMSE) between them,

$$\epsilon = \left(\sum_i \frac{\int |y_i(t) - y_{\text{test},i}(t)|^2 dt}{T \text{Var}[y_{\text{test},i}(t)]} \right)^{1/2}, \quad (2.12)$$

where the integral is evaluated on an interval of length T , and the sum i is over all components of $\mathbf{y}_{\text{test}}(t)$. If $\epsilon = 0$, then the two signals are identical during the interval being evaluated, and any difference between them will result in $\epsilon > 0$.

For RCs trained to do inference, this NRMSE is calculated across the entire testing signal, regardless of length. However, for forecasting this presents a problem. An RC in forecasting mode is autonomous, after the warm-up period, and receives no ongoing external input as it would in an inference task. The testing output may diverge quickly from the reference system. Indeed, if the RC is trained to forecast a chaotic system, good forecasting requires the forecast to diverge eventually. In these situations, the testing error is calculated only for a period of $T = 1/\lambda_{\text{max}}$, where λ_{max} is the largest Lyapunov exponent of the system the RC is forecasting. In this dissertation I will refer to the arbitrary- T error as ϵ , and the one Lyapunov period error as ϵ_1 .

Using ϵ_1 as a testing metric for forecasting RCs is common, but still has problems [11, 13]. The largest issue is that the specific value of ϵ_1 will depend strongly on which specific input signal $\mathbf{u}_{\text{test}}(t)$ is used for testing. If the input signal passes very near an unstable state of the input system, very small errors in the RC's forecasting are amplified dramatically. In these cases, ϵ_1 is not a useful measure of how well the RC has learned the input system. Examples of this are given in Chapter 3.

To address these issues, I instead propose a new metric calculated from m forecasts with many different example inputs. I calculate an ϵ_1 value for each forecast, and then combine

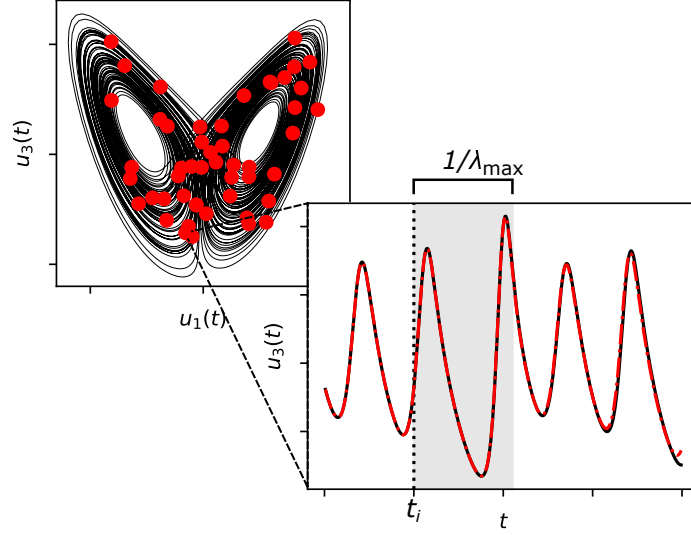


Figure 2.2: Summary of the forecasting evaluation method. I calculate errors $\epsilon_{1,i}$ at times t_i , marked on the attractor as red dots, and combine them into a single error $\tilde{\epsilon}$. Before each t_i (dotted vertical line), the reservoir is integrated with Eq. (2.1) and listening to the input. After t_i , the reservoir is integrated with Eq. (2.7), and runs autonomously. The reservoir’s prediction (dotted red line) must eventually diverge from the true system (solid black line). I calculate each $\epsilon_{1,i}$ only during a single Lyapunov period after forecasting begins, marked here in gray.

them into one final metric

$$\tilde{\epsilon} = \left(\frac{1}{m} \sum_{i=1}^m \epsilon_{1,i}^2 \right)^{1/2}. \quad (2.13)$$

This method is summarized in Fig. 2.2. Using $\tilde{\epsilon}$ averages out the chance encounters with unstable states, and as I will show in Chapter 3, is a more reliable indicator of a forecasting RC’s performance.

For forecasting, a very useful qualitative indicator of an RC’s ability to learn the input system is to use the RC in autonomous mode to produce a very long forecast. This forecasted signal will necessarily diverge from the true system, but both the forecast and the true system can then be plotted side-by-side in state space. A well-trained RC forecast will reproduce the shape of the input system’s attractor, and these plots can reveal immediate problems with the trained RC that metrics like NRMSE sometimes mask. Still, this comparison is completely qualitative – it is imprecise, and not suitable to use in an automated setting. Future research may develop a quantitative way to measure the similarity between

the RC's reconstructed attractor and the true attractor, which would solve the problems with the current NRMSE metric. In the meantime, visual inspection remains a useful sanity check on the NRMSE.

2.7 Summary

Reservoir computers are a simple tool that uses an auxiliary dynamic reservoir to transform its input. They can be trained on example data, and that training is done very quickly using linear regression tools. Once trained, RCs can be used in inference tasks, or in an autonomous mode to perform system forecasting.

The RC approach is robust to the choice of reservoir system, however the specific requirements the reservoir must have to function are largely unknown. When an RC fails, it is difficult to know why. Recent work on invertible generalized synchronization is an exciting path towards learning more.

Echo state networks are a network-based reservoir choice that is easy to simulate in software, and replicates more traditional neural network machine learning designs. This approach uses several meta-parameters to randomly construct the internal network. As with the general reservoirs, there is not much concrete guidance on what values these meta-parameters should have.

In Chapter 3, I will explore some of these meta-parameters, and conclude that several guidelines for ESN construction are not necessary. I will demonstrate several RCs with very simple internal networks that nonetheless successfully replicate the climate of a chaotic dynamic system.

Chapter 3

LOW-CONNECTIVITY RESERVOIRS

The choice of ESN meta-parameters that best fits a given task is difficult to identify. In the absence of strong guiding rules, a practical approach is to treat it as an optimization problem. In this chapter, I explore using ESNs on the chaotic system forecasting task for the Lorenz '63 system, the Rössler system, and the double-scroll circuit system described in Appendix A. This exploration is guided by optimization – by trying to discover the meta-parameters that result in the best averaged NRMSE $\tilde{\epsilon}$ during forecasting for the given system. This exploration leads to a surprising discovery: that ESNs with simple internal structure perform as well as their more complicated siblings. In the process, I find concrete evidence that my new averaged $\tilde{\epsilon}$ metric is a more reliable indicator of forecasting quality than ϵ_1 .

In particular, I am interested in the recurrent connectivity k . In software simulations, such as those described in this dissertation, the connectivity k has only a slight impact on the speed of the simulation. However, in hardware network implementations, each connection has an associated cost. For autonomous Boolean networks implemented on FPGAs, every network connection is built from a long chain of delay nodes [14]. These delays often compose the majority of the resource budget of the hardware network. Although the results in this chapter are all software simulation, the particular focus on the k parameter is done with an eye towards hardware implementations.

Many techniques have been used previously for ESN meta-parameter optimization, such as grid search [21] and gradient descent [26]. However, both of these approaches have

drawbacks. Grid search quickly becomes intractable when the number of dimensions grows, and even after fixing the two functional meta-parameters f and $\mathbf{g}$, there are 5 dimensions. Compounding this problem, the ESN construction is a random process, and so the resulting measured $\tilde{\epsilon}$ for a given set of meta-parameters is a random variable. To obtain a meaningful result, each set of meta-parameters needs to be tested multiple times. For even a coarse grid search, with 10 trials and 10 points per meta-parameter, this already means one million trial RCs. Gradient descent suffers from the randomness of ESN construction as well, for the same reasons. It is also unsuitable for use with discrete parameters such as k .

One way to mitigate these problems with random ESN construction is to modify the ESN construction method outlined in Section 2.4 to front-load the random choices. For example, once the recurrent weight matrix W_r has been constructed, the parameter ρ_r can be changed without any random process at all. By cleverly factoring out the random parts of ESN construction, it becomes possible to use methods like grid search and gradient descent without the extra step of multiple random trials. Ultimately, however, this is optimizing only a single random instantiation of an ESN, and provides little insight into how the fully random ESN will perform.

In this chapter, I use Bayesian optimization [27, 28], as implemented by the `skopt` Python package [29], to explore which meta-parameters result in the lowest forecasting error $\tilde{\epsilon}$ for the three example systems. Bayesian optimization deals well with both noise and integer parameters like k , is more efficient than grid search [28], and works well with minimal tuning.

3.1 Bayesian Optimization

Bayesian optimization is a technique that finds the minimum of an objective function $h(\mathbf{x})$, in this case the measured error $\tilde{\epsilon}$ of an ESN with meta-parameters $\mathbf{x}$, by repeated evaluation. The process starts with a prior distribution of functions, representing the algorithm’s knowledge of the function h . As each new point $\mathbf{x}$ is evaluated, the algorithm incorporates this new knowledge and updates its prior distribution. At the start of the optimization, points

	Parameter	min		max
γ	Leak Rate	7	–	11
σ	Input Connectivity	0.1	–	1.0
ρ_{in}	Input Scale	0.3	–	1.5
k	Recurrent Connectivity	1	–	5
ρ_r	Recurrent Scale	0.3	–	1.5

Table 3.1: Range of meta-parameters searched using Bayesian optimization.

are chosen randomly. This is the exploration phase, where the Bayesian optimizer gathers data to improve its prior distribution. After exploration, the optimizer then evaluates points $\mathbf{x}$ with the most expected improvement over the best known minimum so far.

This algorithm has a number of drawbacks. Primarily, it is complicated. The above summary only scratches the surface, and already suggests a number of parameters to the optimization algorithm that can be tuned, such as the initial prior distribution of functions. This complication makes it difficult to directly claim that any minimum found is the true minimum, and so I emphasize that this algorithm is used here for exploration. I am interested in ESNs that perform well, and what they have in common, but not in claiming any ESN as the best performer possible. For this purpose, the unopinionated defaults of the `skopt` package suffice.

The complexity of this optimization algorithm also manifests in run time. After the exploration phase, calculating the next set of trial parameters $\mathbf{x}$ involves minimizing an internal function calculated from the current prior distribution. This can be quite computationally expensive, and so Bayesian optimization is only suitable for use when the objective function $h(\mathbf{x})$ is expensive enough to justify the extra work to reduce how often it is evaluated. This makes it a good fit here for minimizing prediction error $\tilde{\epsilon}$ as this involves building, training, and testing an RC each time.

In this chapter, the Bayesian algorithm repeatedly generates a set of meta-parameters to test within the ranges listed in Table 3.1. Larger ranges require a longer optimization time. I selected these ranges to include values that existing ESN design heuristics would choose, while also allowing exploration outside those ranges without a prohibitively long

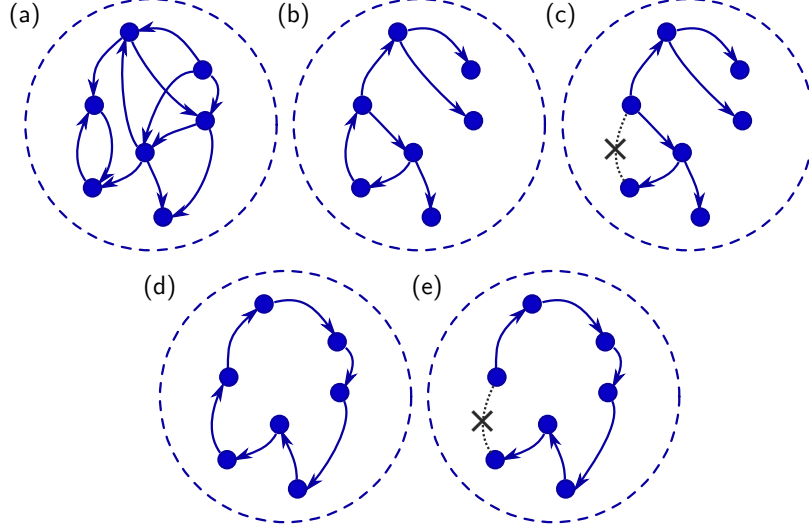


Figure 3.1: The five reservoir structures tested. Only internal reservoir connections are pictured. Connections to the reservoir computer input, or to the output layer are not shown. (a) A general, fixed in-degree network, here pictured with $N = 7$ and $k = 2$. (b) A $k = 1$ network with a single connected component. (c) A $k = 1$ network with the single cycle cut at an arbitrary point. (d) A *simple cycle reservoir*. (e) A *delay line reservoir*.

runtime.

During the optimization process, I discover that the optimizer often finds ESNs with an input connectivity of $k = 1$ that perform as well as ESNs with a higher k . Such networks have an interesting and simple network structure, and also suggest other simple structures for comparison.

3.2 Structure of Low-Connectivity Reservoirs

First, networks generated with $k = 1$ often have multiple disconnected components. For higher k , this is still possible but relatively rare, as k controls the number of opportunities for components in the network to connect. Disconnected components in the network essentially act as smaller ESNs operating in parallel. This is an interesting line of research [30], but not one I investigate further here. In the interest of a more equitable comparison between $k = 1$ networks and $k > 1$, I discard and regenerate any networks with more than a single connected component.

A $k = 1$ network with only a single connected component must also contain only a single directed cycle. Starting from any node in the network and working backwards through the single input connection each node must have will always lead to a single directed cycle, and it is not possible for two such cycles to exist in the network unless they are disconnected from each other. This severely limits how recurrence can occur inside a $k = 1$ network compared to higher- k networks. Every node in a $k = 1$ network is either part of this core cycle, or part of a directed tree branching off from this cycle, as depicted in Fig. 3.1 (b).

Inspired by the high performance of this simple structure, I also investigate $k = 1$ networks when the single cycle is cut at a random point. This turns the entire network into a tree, as in Fig. 3.1 (c). These networks retain the random weights they have before the single cycle is cut.

Finally, I also investigate reservoir networks that consist entirely of a cycle or a ring with identical weights with no attached tree structure, depicted in Fig. 3.1 (d), and networks with a single line of nodes (a cycle that has been cut), in Fig. 3.1 (e). These choices are inspired by the $k = 1$ networks, but have also been researched previously and are known as *simple cycle reservoirs* and *delay line reservoirs*, respectively [21].

In total, there are five network structures I investigate:

- (a) general construction with unrestrained k ,
- (b) $k = 1$ with a single cycle,
- (c) $k = 1$ with a cut cycle,
- (d) single cycle, or *simple cycle reservoir*,
- (e) single line, or *delay line reservoir*.

Both the $k = 1$ cut cycle networks (c) and line networks (e) are rescaled to have a fixed ρ_r before the cycle is cut. However, after the cycle is cut, they both have $\rho_r = 0$. As I demonstrate in this chapter, these simpler network structures have the capability to perform as well as their more complicated counterparts, despite the fact that (c) and (e) have no recurrence and a spectral radius nowhere near $\rho_r = 1$.

3.3 ESN Symmetries and their Consequences

An ESN in autonomous forecast mode, described by Table 2.1 (b), has an inversion symmetry about the origin. That is, if $\mathbf{r}(t)$ is a solution to this equation, so is $-\mathbf{r}(t)$. For an output layer with an identity read-out function $\mathbf{g}$, this means that if $\mathbf{y}(t)$ is a possible output of the trained RC, then so is $-\mathbf{y}(t)$. If the underlying system the ESN is trained to forecast also exhibits this symmetry, this is not a problem. In fact, designing the internal reservoir to match symmetries with the input system can dramatically improve ESN performance [31].

However, if the input system does not have this symmetry, this can be an issue. If at any time the autonomous forecast strays near $\mathbf{r} = 0$, the solution can hop over the symmetry to the other side, and begin producing an output $\mathbf{y}(t)$ that is flipped through the origin. This problem is exacerbated if the input signal $\mathbf{u}(t)$ is normalized to have mean zero, a common practice.

This problem is well-known, and a common fix [11, 32] is to use a nonlinear read-out function

$$g_i(\mathbf{r}) = \begin{cases} r_i & \text{if } i \leq N/2, \\ r_i^2 & \text{if } i > N/2. \end{cases} \quad (3.1)$$

This squares half of the node values before being passed to the output layer, and breaks the inversion symmetry in the ESN equation while keeping the number of features accessible to the output layer the same.

This problem can be difficult to diagnose if the input system has a partial inversion symmetry. For example, the Lorenz '63 system in Eq. (A.1) has an $(x, y) \rightarrow (-x, -y)$ symmetry, and so the complete inversion of the Lorenz attractor looks exactly the same as a flip along the z axis. This is misleading, as the problem has nothing to do with the Lorenz z variable specifically, and this confusion prevented a full understanding of the problem until recently [32]. Figure 3.2 provides an example of the forecasting failure characteristic to the identity read-out function when applied to the Lorenz '63 system. The ESN does a poor job reconstructing the Lorenz attractor, but it also reproduces this attractor reflected through the origin. This mirror attractor is not a feature of Lorenz '63, is not present in the training

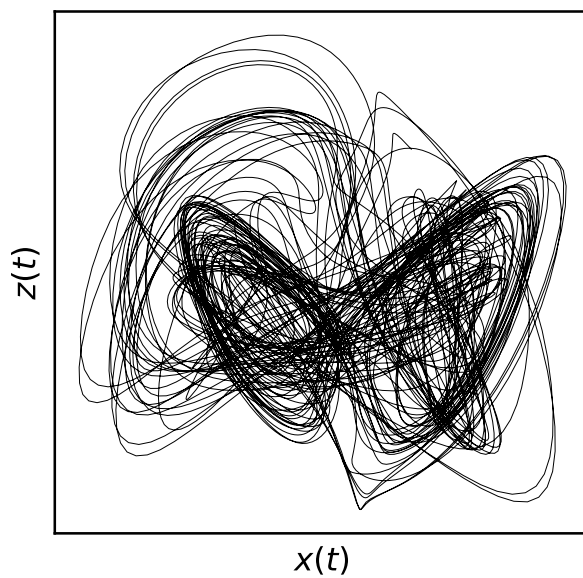


Figure 3.2: The forecasted attractor from an ESN with full inversion symmetry trained on the Lorenz '63 system, using identity read-out, in the x/z plane. Compare against Fig. A.2, and note the two deformed images of the butterfly attractor on top of each other, with one flipped along the z axis. Breaking this inversion symmetry in the read-out function is enough to prevent this kind of failure entirely.

data used to produce Fig. 3.2, and is entirely due to the symmetry of the ESN equations. Using the read-out function in Eq. (3.1) removes this symmetry.

For consistency, I use this *quadratic read-out* for all three example input systems used in this chapter, even if the inversion symmetry does exist in the input system as it does for the double-scroll circuit.

3.4 Training, Forecasting, and Evaluation

Now that I have identified the five networks structures of interest, I use the Bayesian optimization algorithm to identify networks in each class that perform well on the system forecasting task. I optimize the meta-parameters independently for each of the three example input systems: Lorenz '63, Rössler, and the double-scroll circuit. I describe all three systems in Appendix A.

For each input system and for each network structure, the Bayesian algorithm performs random exploration for 50 iterations, and then optimization for 50 iterations, resulting in 100 iterations in total. At each iteration of the algorithm, the optimizer constructs a single random instantiation of an ESN with $N = 100$ nodes and the chosen meta-parameters, trains it, and measures its forecasting performance with the metric $\tilde{\epsilon}$ defined in Eq. (2.13). Since the exploration phase of the optimization is random, I repeat this entire process 20 times to estimate the variance in the performance of reservoirs optimized by this method.

To ensure that results are comparable across all three systems, I normalize their components to have 0 mean and unit variance. In addition, I rescale the time axes of all systems so that their maximum positive Lyapunov exponent matches that of the Lorenz system, $\lambda_{\max} = 0.9056$ [33].

To train the ESN, I integrate Eq. (2.8) with the quadratic readout in Eq. (3.1), coupled with the chosen input system, via the hybrid Runge-Kutta 5(4) [34] method from $t = 0$ to 300. This produces an ESN response $\mathbf{r}(t)$. I then divide this interval into three ranges:

- $t = 0 - 100$: the warm-up period,
- $t = 100 - 200$: the training period,

- $t = 200 - 300$: the testing period.

As discussed in Section 2.1, the warm-up period is used to ensure the later times do not depend on the specific initial conditions of the ESN. I divide the rest into a training period, used only during training, and a testing period, used later only to evaluate the ESN performance.

I then determine W_{out} via ridge regression, as in Eq. (2.4), where the sum ranges over the training period. The ridge parameter α could be included among the meta-parameters to optimize. However, unlike the other meta-parameters, modifying α does not require re-integration and can be optimized with simpler methods. I select an α from among 10^{-5} to 10^5 by leave-one-out cross-validation, which has an extremely efficient implementation for ridge regression [35]. Using the computationally cheaper cross-validation for α instead of the Bayesian algorithm helps to keep the optimizer’s computational effort focused on those parameters that most benefit from it.

To evaluate the performance of the trained ESN, I use it to perform autonomous forecasting using the autonomous equation Table 2.1 (b). I choose 50 times t_i spaced evenly within the testing period. For each t_i , I initialize the reservoir state to $\mathbf{r}(t_i)$, and then integrate forward with the autonomous equation for one Lyapunov period, between $t = t_i$ and $t = t_i + 1/\lambda_{\text{max}}$. This produces a reservoir forecast during these times, from which I calculate an error $\epsilon_{1,i}$ as described in Eq. (2.12). Finally, I combine these 50 errors into a single $\tilde{\epsilon}$, as in Eq. (2.13), that represents the average ability of the ESN to forecast accurately at any point on the input system attractor.

Using $\tilde{\epsilon}$ and not simply ϵ_1 is even more important in an optimization setting than usual, as searching for ESNs with the lowest ϵ_1 risks producing ESNs that are only good at forecasts near t_1 , but otherwise perform poorly in reproducing the attractor of their input system. It can also waste time, as the optimization algorithm explores areas of the parameter space it believes perform well, but in fact only perform coincidentally well at t_1 . Figure 3.3 depicts two common ways for an ESN to fail to replicate the true Lorenz attractor. In Fig. 3.3 (i), the ESN learns the right lobe of the attractor reasonably well, as this is where t_1 resides,

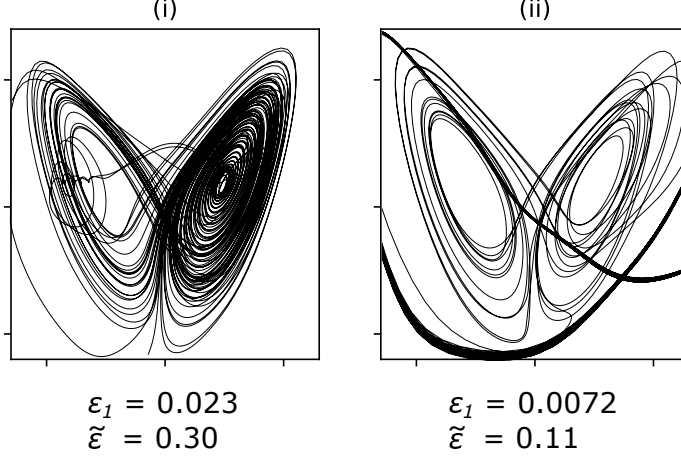


Figure 3.3: Two examples of ESNs that fail to reproduce the Lorenz attractor, but produce a low ϵ_1 . Compare with the true attractor in Fig. A.2. Network (i) fails to learn the attractor, overemphasizing the right lobe in which t_1 resides. Network (ii) matches the attractor well early on, but as the prediction lengthens, it falls into a periodic orbit (thick black line). Both (i) and (ii) show a promising low ϵ_1 , but the averaged $\tilde{\epsilon}$ measure more accurately captures their failure to learn the Lorenz attractor.

but quickly exhibits non-Lorenz behavior in the left lobe. In Fig. 3.3 (ii), the ESN performs well in both lobes for a short period of time before settling in to a periodic orbit that does not match Lorenz at all. However, both produce a good short-term forecast near t_1 . In particular, network (ii) in Fig. 3.3 has a lower ϵ_1 than any of the optimized reservoirs I found despite its obvious failure to learn the long-term Lorenz attractor.

3.5 Results

3.5.1 Forecasting Lorenz '63

First, I run reservoir networks of all five structures through 100 iterations of the Bayesian algorithm using the Lorenz system as input, and record the best-performance RC for each structure according to the metric $\tilde{\epsilon}$. These reservoirs, and the meta-parameters that generated them, are reported in Table 3.2. I estimate the error on $\tilde{\epsilon}$ with the standard deviation of $\tilde{\epsilon}$ after repeating the optimization process 20 times.

When optimized, all reservoir structures perform well. In particular, the simpler structures all perform almost as well as the general- k structure. They often lie within one, or at

		Lorenz				
	Structure	$\tilde{\epsilon}$	γ	σ	ρ_{in}	ρ_r
(a)	Any k ^a	0.022 ± 0.004	7.7	0.81	0.37	0.41
(b)	$k = 1$ with cycle	0.024 ± 0.005	10.9	0.44	0.30	0.30
(c)	$k = 1$ no cycle	0.028 ± 0.005	7.2	0.78	0.30	0.30^b
(d)	cycle	0.023 ± 0.008	7.9	0.17	0.58	0.30
(e)	line	0.024 ± 0.003	10.6	0.79	0.30	0.45^b

Table 3.2: Best reservoir computers of each reservoir structure, after 100 iterations of the Bayesian optimization algorithm using the Lorenz system as input. The meta-parameters chosen by the algorithm are shown on the right. The simpler structures (b) – (e) all perform nearly as well as the general structure (a).

^aAfter optimization, $k = 3$.

^b ρ_r measured before cycle is cut. Afterwards, $\rho_r = 0$.

most two standard deviations from structure (a). This is despite the fact that structures (c) and (e) both have $\rho_r = 0$ and no recurrent connections within the network. The other structures have $\rho_r \ll 1$. Previous work has already demonstrated that ESNs with low or zero spectral radius can still function [11, 21]. These results act as additional counterexamples to the heuristic that ESNs should have recurrence and $\rho_r \approx 1$ [17].

3.5.2 ESN Variation

These best-observed ESNs are not representative of a typical ESN. I use the meta-parameters to guide the random input connections and connections within the network, and so even once the meta-parameters are fixed, constructing the network is a random process. Not all networks with a fixed structure type and meta-parameters will perform the same.

To explore this variation, I generate and evaluate 200 networks of each structure on the Lorenz system using the optimized meta-parameters in Table 3.2. For all five structures, as the measured $\tilde{\epsilon}$ increases, the quality of the reproduced attractor decreases gradually. On manual inspection of the attractors, the reason for this decrease can be divided into three qualitative regions. For $\tilde{\epsilon} < 0.3$, the RCs reproduce the Lorenz attractor consistently. Failures still rarely occur, but they always reproduce part of the attractor before falling into a fixed point or periodic orbit. In this region, small differences between the true attractor

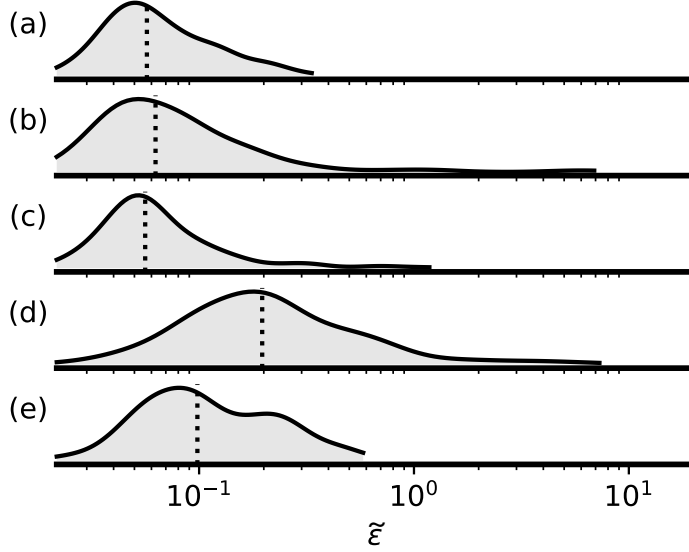


Figure 3.4: Performance for each network structure, evaluated for the Lorenz system at the meta-parameters listed in Table 3.2. Each is visualized as a Gaussian kernel density estimation in $\log_{10} \tilde{\epsilon}$ with a bandwidth of 0.35 [36], which can be interpreted as a probability distribution. Using a narrower bandwidth does not reveal additional features. A vertical line marks the median. Note that structures (b) – (e) have very long tails, and can produce networks that perform very poorly in comparison to (a). However, all five have the capability to produce well-performing networks.

and the reproduced attractor contribute more to $\tilde{\epsilon}$ than outright failure. For $0.3 < \tilde{\epsilon} < 1.0$, RCs always fail to reproduce the attractor, though they will still always reproduce a portion of it before failing. Examples of these failures are provided in Fig. 3.3. Above $\tilde{\epsilon} > 1.0$, these failures become catastrophic, and no longer resemble the Lorenz attractor at all. A more quantitative description of these regions is one line of possible future research.

Though the optimized best-performing networks of each structure show very little performance difference, the differences between them become more apparent when we compare the probability distribution of $\tilde{\epsilon}$ for each topology. These distributions are shown in Fig. 3.4.

A well-performing network with arbitrary k (a) is a much more likely outcome than a well-performing network with a single cycle (d). However, the performance of arbitrary k networks (a) is very similar to that of tree-like networks (c). Though (c) has a longer tail on the high end, the simpler structure of the reservoir may be appealing for hardware RCs.

The distribution of performance for each structure can be a deciding factor if network

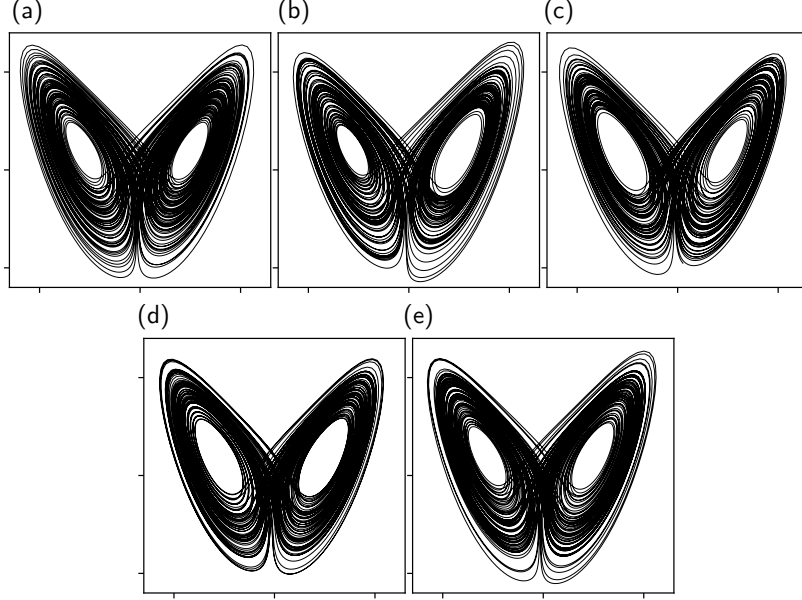


Figure 3.5: Lorenz attractor plots in the x/z plane for long-term free-running predictions for each network structure in Table 3.2. Compare with true Lorenz attractor in Fig. A.2, and the failed reservoirs in Fig. 3.3.

construction and evaluation is expensive, as it might be in hardware. In software, though, I find the best-performing networks in Table 3.2 after only 100 trials. A hardware design can still benefit from the simpler structures (b) – (e) despite their very wide performance distributions if the creation of the evaluation of the design can be automated to test many candidate reservoirs, as on an FPGA [14].

There may also be a benefit in software. The simpler structures are represented by weight matrices W_r in very simple forms. Structure (c) can always be represented as a strictly lower-diagonal (or upper-diagonal) matrix, and (d) – (e) can be represented with non-zero entries only directly below the main diagonal. Software simulations can take advantage of this structure to speed up integration of Eq. (2.8).

3.5.3 Reproducing the Lorenz '63 Attractor

For each structure I produced a free-running prediction of the Lorenz system for 100 time units using the best-performing RC. I use these predictions to produce an attractor as shown in Fig. 3.5. All five optimized ESNs reproduce the Lorenz attractor well. Though

Structure		Double Scroll $\tilde{\epsilon}$	Rössler $\tilde{\epsilon}$
(a)	Any k	0.029 ± 0.006	0.017 ± 0.005
(b)	$k = 1$ with cycle	0.033 ± 0.007	0.020 ± 0.007
(c)	$k = 1$ no cycle	0.033 ± 0.008	0.018 ± 0.006
(d)	cycle	0.033 ± 0.007	0.018 ± 0.006
(e)	line	0.037 ± 0.01	0.019 ± 0.015

Table 3.3: Result of optimizing reservoirs with the Bayesian algorithm over 100 iterations, on both the Rössler system and the double-scroll system. All five structures can perform equally well at both systems.

comparing these plots by eye is not quantitative, I find them qualitatively useful: an RC that fails to reproduce the Lorenz attractor by eye is unlikely to have a low $\tilde{\epsilon}$ compared to one that does, but visual inspection of the reproduced attractor can catch rare catastrophic failures that are not reflected in $\tilde{\epsilon}$.

3.5.4 Forecasting Rössler and the Double-Scroll Circuit

To explore whether these structures remain equally effective at tasks beyond forecasting the Lorenz system, I run all five network structures through 100 iterations of the Bayesian algorithm for both the Rössler and the double-scroll systems. As with the Lorenz system, I estimate the errors on these results by repeating the process 20 times each. These results are reported in Table 3.3.

The results for the double-scroll and Rössler systems agree well with those for Lorenz. All five structures optimize reliably with the Bayesian algorithm, and all perform similarly when optimized. Optimizing a network to reproduce either system will almost always work after only 100 iterations, regardless of networks structure.

3.5.5 Cross-task Performance

As RC training is computationally cheap, one advantage to RCs is that a single reservoir can be quickly re-used on many different tasks by re-training W_{out} . To evaluate whether this is possible with these optimized networks, I take each of the 20 networks optimized for the Lorenz system and re-train W_{out} to instead reproduce the double-scroll circuit system.

		Double Scroll	
	Topology	$\tilde{\epsilon}$	$\tilde{\epsilon}_{\min}$
(a)	Any k	0.43 ± 1.2	0.028
(b)	$k = 1$ with cycle	0.30 ± 0.5	0.048
(c)	$k = 1$ no cycle	0.37 ± 0.8	0.032
(d)	cycle	0.17 ± 0.2	0.056
(e)	line	0.22 ± 0.3	0.058

Table 3.4: Result of re-using reservoirs optimized for Lorenz prediction to perform double-scroll prediction. The minimum error encountered across all 20 reservoirs of each topology is reported as $\tilde{\epsilon}_{\min}$.

Every other part of the RC is left unchanged. I then evaluate how accurate this prediction is using the metric $\tilde{\epsilon}$. These results are summarized in Table 3.4.

In general, these reservoirs perform poorly on this new task. However, there is extremely high variation. Even though they are optimized to perform Lorenz forecasting, many of these reservoirs are still able to reproduce the double-scroll attractor. Moreover, the best performers in each category approach the performance of reservoirs optimized specifically for the double-scroll system. This indicates that it is possible to find a single reservoir in any of these topologies that works well on more than one system. The Bayesian optimization algorithm may be able to find these reservoirs more reliably if the metric $\tilde{\epsilon}$ is modified to reward RCs that perform well on many systems.

3.6 Conclusion

I find that Bayesian optimization of RC meta-parameters is a useful tool for creating high-performance ESNs quickly. I also find that allowing the optimizer to explore areas of the parameter space that are typically excluded in other optimization studies can lead to interesting, effective, and possibly surprising ESN designs such as those presented here.

For this procedure to be effective, I find that evaluating the RC performance at many points along the attractor and averaging, rather than evaluating only at a single point, encourages the optimization algorithm to find networks that reproduce the climate of the input system. Using this evaluation method helps direct the optimizer away from reser-

voirs that perform good short-term forecasting, but at only one point on the input system attractor.

One surprising outcome of this optimization procedure is finding reservoirs that perform well even with no recurrent connections and $\rho_r = 0$. Though some reservoirs of this kind have been explored previously and shown to work [11, 21], the heuristics remain common in reservoir design. I present these networks as concrete examples that provide evidence these heuristics are not unbreakable rules, and that breaking these rules does not need to come at the cost of forecasting performance.

In greater detail, I find reservoir networks with very low internal connectivity that perform at least as well as their higher-connectivity counterparts. A network with only a single internal cycle, or even no cycle at all, can perform as well as those with many recurrent cycles. These simpler structures manifest as simpler weight matrices, which can result in faster integration in software. In a hardware environment where connections between nodes have a cost, or recurrence is difficult to implement, these network structures may also have a direct benefit.

Though the best of these low-connectivity networks perform as well as the more complicated reservoirs, they also tend to perform worse on average. However, searching for the best-performing instance of these reservoirs can be done in few trials, and may be feasible for hardware reservoirs that can be constructed and evaluated in an automated way.

During this work, I have discovered many interesting lines of future research. First, the metric $\tilde{\epsilon}$ can be further evaluated by comparing the output of the reservoir predictions to the true system attractor using a new metric for attractor overlap [37]. This overlap metric may also be used to quantify my qualitative observations of different failure modes in regions of the $\tilde{\epsilon}$ metric. Second, these results show that these very low connectivity reservoirs perform well in the narrow context of software-based, chaotic system forecasting. Future work can explore whether these results hold for other reservoir computing tasks such as classification or state inference, and whether it is possible to find reservoirs that perform well on a variety of tasks simultaneously by modifying the metric $\tilde{\epsilon}$ to encourage generalization. Third, these results may also generalize to RCs with hardware networks, where the simpler reservoir

designs would allow for more efficient and faster operating hardware RCs.

At the time the work in this chapter was conducted, there were known results that prove that a linear network architecture with time-independent nodes, the discrete-time NARX networks, can simulate fully-connected networks [38]. A similar proof for RCs would explain why there is no significant difference in the best-performing reservoirs in each structure type. My discovery that a line network (e) can perform as well as a general- k network (a) suggests that the network's role in the RC may only be to produce a variety of time delays for the output layer to draw on.

In the time since, such a proof has been discovered. In the next chapter, I will introduce nonlinear vector auto-regressions (NVARs), an existing machine learning tool that is proven to be mathematically equivalent to ESNs under specific conditions, and discuss how this relationship between NVARs and RCs has advanced the understanding and application of both.

Chapter 4

RCs WITHOUT RESERVOIRS: NONLINEAR VECTOR AUTO-REGRESSIONS

In 2021, Erik Bollt proved [39] that a completely linear ESN is mathematically equivalent to an existing tool known as a *vector auto-regression*, or VAR. He subsequently proved that an ESN with a quadratic output nonlinearity is likewise equivalent to a *nonlinear VAR*, or NVAR.

In this chapter, I provide a brief outline of this equivalence proof, beginning with a discussion of the behavior of fully linear ESNs as well as their drawbacks. Later, I describe the VAR and NVAR methods. This provides background information and motivation for the original work presented in Chapter 5.

Although output-nonlinear ESNs and NVARs are mathematically equivalent, the conditions of their equivalence raise some practical concerns when it comes to implementation. In this chapter discuss these concerns, as well as the relative merits of the ESN and NVAR approaches. In addition, I discuss how the NVAR approach benefits from reinterpretation within the RC framework.

In Chapter 5 I build on the information provided here to demonstrate the NVAR approach in practical applications, and address these concerns about the applicability of Bollt’s proof.

4.1 Solutions to Linear ESNs

4.1.1 Prediction

A fully linear ESN, with both the activation function f and the read-out function $\mathbf{g}$ set to the identity function, has very simple solutions. As an example, I will take the discrete-time linear ESN equation in prediction mode from Table 2.1 (d),

$$\mathbf{r}(t + \Delta t) = (1 - \gamma\Delta t)\mathbf{r}(t) + \gamma\Delta t(W_r \mathbf{r}(t) + W_{\text{in}} W_{\text{out}} \mathbf{r}(t)), \quad (4.1)$$

$$\mathbf{y}(t + \Delta t) = W_{\text{out}} \mathbf{r}(t + \Delta t). \quad (4.2)$$

The linear coefficients of $\mathbf{r}(t)$ can be collected into a single matrix

$$W \equiv (1 - \gamma\Delta t)I + \gamma\Delta t(W_r + W_{\text{in}} W_{\text{out}}), \quad (4.3)$$

simplifying the Eq. (4.1) to

$$\mathbf{r}(t + \Delta t) = W \mathbf{r}(t). \quad (4.4)$$

Almost all square matrices of complex numbers are diagonalizable. If W is an $N \times N$ matrix, diagonalizing W yields N uncoupled time evolution equations $r_i(t + \Delta t) = w_i r_i(t)$ where w_i may be complex. Solutions to this equation are very simple,

$$r_i(k\Delta t) = w_i^k r_i(0). \quad (4.5)$$

This means the dynamics of the ESN amount to discrete-time complex waves, possibly with an exponential growth or decay over time. This is all the RC output layer has to draw from to construct the output $\mathbf{y}(t)$. For the forecasting task, a sum of sine waves of different frequencies can perform adequately well for short-term prediction [39], as this effectively amounts to a discrete Fourier approximation. However, for long term attractor reconstruction and the reproduction of global properties of the input system, this linear ESN *must* eventually fail on anything but the simplest input system. The most direct demonstration of this failure is that this linear ESN cannot predict any fixed point of the input system except $\mathbf{y}(t) = 0$. It is possible to shift this fixed point to another location by

adding a constant term to Eq. (4.4), but this change can only move the fixed point around, not add new ones.

To have any hope of reconstructing the underlying system attractor, as demonstrated with ESNs in Chapter 3, an ESN must have some nonlinear component, either in the activation function f or the read-out function $\mathbf{g}$. Still, a completely linear ESN is mathematically easy to work with, and indeed the RC/NVAR equivalence proof begins by looking at the behavior of a discrete-time linear ESN in inference mode.

4.1.2 Inference

For ESNs in inference mode, from Table 2.1 (c),

$$\mathbf{r}(t + \Delta t) = (1 - \gamma\Delta t)\mathbf{r}(t) + \gamma\Delta t(W_r \mathbf{r}(t) + W_{\text{in}} \mathbf{u}(t)), \quad (4.6)$$

$$\mathbf{y}(t + \Delta t) = W_{\text{out}} \mathbf{r}(t + \Delta t). \quad (4.7)$$

The coefficients of $\mathbf{r}(t)$ and $\mathbf{u}(t)$ can be collected into two matrices,

$$A \equiv (1 - \gamma\Delta t)I + \gamma\Delta t W_r, \quad (4.8)$$

$$B \equiv \gamma\Delta t W_{\text{in}}. \quad (4.9)$$

Equation (4.6) then simplifies to

$$\mathbf{r}(t + \Delta t) = B \mathbf{u}(t) + A \mathbf{r}(t). \quad (4.10)$$

This equation is recursive; expanding it out into the past yields

$$\mathbf{r}(t + \Delta t) = B \mathbf{u}(t) + AB \mathbf{u}(t - \Delta t) + A^2 \mathbf{r}(t), \quad (4.11)$$

$$\mathbf{r}(t + \Delta t) = \sum_{j=0}^{\infty} A^j B \mathbf{u}(t - j\Delta t). \quad (4.12)$$

Equation (4.12) states that the value of $\mathbf{r}(t + \Delta t)$ is constructed from a linear combination of time-delay taps of the input $\mathbf{u}(t)$, extending infinitely into the past. In combination with Eq. (4.7), this means the overall output $\mathbf{y}(t)$ is a linear combination of time-delay taps of the input $\mathbf{u}(t)$. This is known as a vector auto-regression.

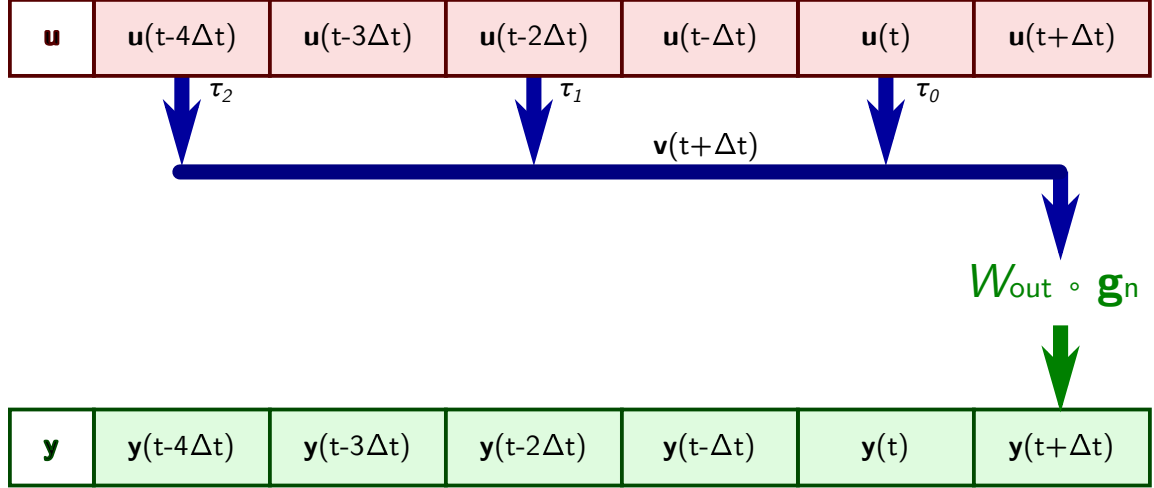


Figure 4.1: Summary of the (N)VAR method. Many time-delay taps τ_i of the discrete-time signal $\mathbf{u}(t)$ (top, red) are concatenated into the tap vector $\mathbf{v}(t)$ (middle, blue). Here, there are three taps $\tau_0 = 0$, $\tau_1 = 2$, and $\tau_2 = 4$. These taps are then passed through a possibly nonlinear function $\mathbf{g}_n$ and combined linearly by the output matrix $\mathbf{W}_{\text{out}}$ to produce the next value of the (N)VAR's output $\mathbf{y}(t)$ (bottom, green). For a linear VAR, $\mathbf{g}_n$ is the identity function. To transform a whole time series input, this (N)VAR process slides along the time axis from left to right.

4.2 Linear Vector Auto-regressions (VARs)

In most cases, the term *vector auto-regression* specifically means a method of producing an output $\mathbf{y}(t)$ from a linear combination of past outputs. However, I generalize this slightly to better mirror the reservoir computer model introduced in Chapter 2.

In this dissertation, a vector auto-regression (VAR) is a method of transforming a discrete-time input signal $\mathbf{u}(t)$ into a discrete-time output signal $\mathbf{y}(t)$ from a linear combination of time-delay taps. Given a list of tap delays τ_i , I construct a tap vector $\mathbf{v}(t)$ via

$$\mathbf{v}(t + \Delta t) = \mathbf{u}(t - \tau_0 \Delta t) \oplus \mathbf{u}(t - \tau_1 \Delta t) \oplus \cdots \oplus \mathbf{u}(t - \tau_{q-1} \Delta t) \quad (4.13)$$

where the operator $\oplus$ is understood to mean vector concatenation. That is, if the input $\mathbf{u}(t)$ has dimension d , and if there are q taps, then the vector $\mathbf{v}(t)$ has dimension dq . Most commonly, $\tau_0 = 0$, which ensures that the VAR has access to the most recent information from $\mathbf{u}(t)$ when producing an output.

The output of the VAR is the linear transformation of this tap vector by the matrix W_{out} ,

$$\mathbf{y}(t + \Delta t) = W_{\text{out}} \mathbf{g}_n(\mathbf{v}(t + \Delta t)), \quad (4.14)$$

where $\mathbf{g}_n$ is the identity function. If $\mathbf{g}_n$ is instead chosen to be nonlinear, this results in a nonlinear VAR, which I discuss in Section 4.3.

This transformation process, from input to output, is summarized in Fig. 4.1. Note the similarity to the RC method: the input signal $\mathbf{u}(t)$ drives an internal state $\mathbf{v}(t)$ / $\mathbf{r}(t)$, which is then linearly transformed into the output $\mathbf{y}(t)$. However, the internal state $\mathbf{v}(t)$ of the VAR is much simpler than that of most reservoirs, because it is constructed directly from the input.

4.2.1 Training and Testing

Once the taps τ_i are selected, the VAR can be trained in much the same way as an RC. The VAR is fed an example input $\mathbf{u}_{\text{train}}(t)$, which produces an example tap vector $\mathbf{v}_{\text{train}}(t)$ via Eq. (4.13). Finally, a W_{out} that best maps this $\mathbf{v}_{\text{train}}(t)$ onto the example output $\mathbf{y}_{\text{train}}(t)$ can be found via ridge regression, exactly as in the RC case.

VARs can also be used for forecasting as well. Analogously to the RC case, the VAR used for forecasting can be trained to reproduce the example input as its output. Since the VAR output $\mathbf{y}(t + \Delta t)$ depends only on $\mathbf{u}(t)$ at time t and earlier, this amounts to one step ahead prediction. Once trained, the VAR can do autonomous prediction by replacing the input $\mathbf{u}(t)$ with the output $\mathbf{y}(t)$ in Eq. (4.13).

For practical reasons, in this dissertation, forecasting VARs have a modified output equation

$$\mathbf{y}(t + \Delta t) = \mathbf{u}(t) + W_{\text{out}} \mathbf{g}_n(\mathbf{v}(t + \Delta t)). \quad (4.15)$$

This effectively changes the VAR from one step ahead prediction to predicting the difference between the last value of the signal and the next, in analogy to a discrete-time integrator. Without this change, the components of W_{out} corresponding to $\mathbf{u}(t)$ are larger than the others. Because the ridge regression used to find W_{out} has a single regularization parameter

α , it works best when all components of W_{out} have about the same expected scale.

Once trained, either as a signal transformation or for prediction, a VAR can be tested in the same ways as an RC. For signal transformations, the VAR output is compared to the true signal, yielding a NRMSE ϵ . For prediction, the VAR can be used to generate multiple predictions for a single Lyapunov time, and the NRMSE ϵ_1 from these predictions is combined into an average error $\tilde{\epsilon}$.

4.2.2 Comparing VARs to ESNs

Working with a VAR, by training it and using it, is very similar to working with an RC. The most important difference is that using a VAR completely sidesteps the issue of choosing an internal reservoir. Building an ESN reservoir is a complicated process that starts with the meta-parameters listed in Table 2.2 and ends with a random realization of a network, usually of 100 nodes or more. In contrast, building a VAR amounts to selecting which taps τ_i to use, and how many. This dramatic reduction in the parameter space makes VARs easier to build and use, and the lack of a random component makes it possible to deterministically specify a VAR’s construction.

By describing the VAR method within the reservoir computing framework, there are a few benefits to the VAR method as well. Previous work with VARs has trained the W_{out} matrix with least squares, while work in the RC field has long used ridge regression as a way to reduce overfitting and encourage generalization. In addition, VARs benefit from the more sophisticated forecasting evaluation methods, such as $\tilde{\epsilon}$, developed for use with RCs.

However, the linear ESN solution in Eq. (4.12) is only completely equivalent to a VAR with an infinite number of time delay taps. VARs with a finite number of taps q approximate a VAR with infinite taps under the right conditions as $q \rightarrow \infty$ [39], but if VARs are to be used as a replacement for RCs, it is critical to know how good this approximation is in practice. If a VAR requires thousands of taps to work, it may still be simpler just to use an ESN.

Finally, these linear VARs suffer from the same drawbacks as the linear ESN. Any practical replacement for a full, nonlinear ESN would have to reproduce the attractors of

the systems they are trained on, as in Chapter 3, and to do that, they need a nonlinearity.

4.3 Nonlinear Vector Autoregressions (NVARs)

The ESN model has two possible sources of nonlinearity: the activation function f and the read-out function $\mathbf{g}$. Most commonly, including in Chapter 3, $f = \tanh$. However, a nonlinear f is mathematically difficult to work with, as it lies in the middle of the ESN's internal state equation. In contrast, the read-out function $\mathbf{g}$ sits only on the output layer. Consider the linear ESN described in Eq. (4.12) modified to have a nonlinear read-out function

$$g_i(\mathbf{r}) = \begin{cases} r_i & \text{if } i \leq N, \\ r_{i-N}^2 & \text{if } N < i \leq 2N. \end{cases} \quad (4.16)$$

where N is the number of nodes in the ESN. That is, $\mathbf{g}(\mathbf{r})$ contains all of the node values, as well as the square of all the node values. This is similar to the read-out function used in Chapter 3, except that it makes no attempt to keep the dimension of $\mathbf{g}(\mathbf{r})$ the same as the dimension of $\mathbf{r}$. By Eq. (4.12), each node value is a linear combination of time-delay taps of $\mathbf{u}(t)$, and so the square of a node value contains only quadratic terms of the form $u_i(t - j\Delta t)u_k(t - l\Delta t)$.

Putting this together, the ESN's output $\mathbf{y}(t)$ can be written in terms of the time-delay tap vector $\mathbf{v}(t)$ in Eq. (4.13), using an infinite number of taps $\tau_i = i$,

$$\mathbf{y}(t + \Delta t) = W_{\text{out}} \mathbf{g}_n(\mathbf{v}(t + \Delta t)). \quad (4.17)$$

The nonlinear function $\mathbf{g}_n(\mathbf{v})$ has all components of $\mathbf{v}$ as well as all quadratic pairs of those components,

$$\mathbf{g}_n(\mathbf{v}) = \mathbf{v} \oplus [\mathbf{v} \otimes \mathbf{v}]. \quad (4.18)$$

Here, the operator $\otimes$ denotes the outer product, and $[X]$ denotes the a flattened vector that contains the upper-triangular components of X . For example, $[\mathbf{v} \otimes \mathbf{v}]$ is a vector that contains as components all unique terms of the form $v_i v_j$.

This construction, a VAR that wraps the delay tap vector in a nonlinear function $\mathbf{g}_n$

before the building the linear combination for the output, is known as a nonlinear VAR (NVAR). This concludes a sketch of Bollt's proof that a linear ESN with quadratic read-out is mathematically equivalent to an NVAR with infinite taps [39]. Because the read-out function appears only on the output, it is not difficult to change the nature of the nonlinear read-out $\mathbf{g}$ in an ESN and derive the corresponding NVAR $\mathbf{g}_n$, and therefore this proof generalizes easily to read-out functions that are more complicated than quadratic.

It is important to note at this point that the ESN's output weights W_{out} and read-out $\mathbf{g}$ are *not* the same as the equivalent NVAR's output weights W_{out} and nonlinearity $\mathbf{g}_{\text{out}}$. In this example, the ESN's $\mathbf{g}$ produces node values and squares of node values, while the NVAR's $\mathbf{g}_n$ produces all linear and quadratic terms it is possible to construct from the tap vector $\mathbf{v}$. The NVAR has quadratic terms that mix both the dimensions of the input $\mathbf{u}(t)$ as well as mix the tap delays τ_i . In a practical application with a finite tap vector, this means that if $\mathbf{v}$ has q taps and $\mathbf{u}$ is dimension d , then $\mathbf{g}_n$ will have $qd + qd(qd + 1)/2$ components. This quadratic dependence on qd can rapidly cause the computational cost of the ridge regression to find W_{out} to be very expensive, as either the number of taps or the input dimension increases.

Putting all the pieces together, the NVAR method for transforming an input $\mathbf{u}(t)$ into an output $\mathbf{y}(t)$ is described by

$$\mathbf{v}(t + \Delta t) = \mathbf{u}(t - \tau_0 \Delta t) \oplus \mathbf{u}(t - \tau_1 \Delta t) \oplus \cdots \oplus \mathbf{u}(t - \tau_{k-1} \Delta t), \quad (4.19)$$

$$\mathbf{y}(t + \Delta t) = W_{\text{out}} \mathbf{g}_n(\mathbf{v}(t + \Delta t)). \quad (4.20)$$

For forecasting tasks, the output Eq. (4.20) is replaced with

$$\mathbf{y}(t + \Delta t) = \mathbf{u}(t) + W_{\text{out}} \mathbf{g}_n(\mathbf{v}(t + \Delta t)). \quad (4.21)$$

Simply by wrapping the tap vector $\mathbf{v}$ in a nonlinear function before the output, the NVAR now has the nonlinearity required to have any hope of reproducing attractors as well as the nonlinear ESNs, and in Chapter 5 I demonstrate that NVARs perform quite well in this role.

	Parameter	Description
τ_i	Tap Delays	the placement of time-delay taps
$\mathbf{g}_n$	Read-out Function	the nonlinearity on the output layer
Δt	Time Step	the discrete time step of both the input $\mathbf{u}(t)$ and the NVAR itself
α	Ridge Parameter	regularization parameter for ridge regression

Table 4.1: Summary of NVAR parameters. Compared to the ESN approach in Table 2.2, the NVAR method is much simpler.

4.4 Practical Considerations

The above sketch of Boltt’s proof demonstrates that a linear ESN with nonlinear readout is equivalent to an NVAR with an infinite number of taps. In practice, though, this method can only be implemented with a finite number of taps. If an NVAR implementation requires thousands of taps to work, it is not a realistic replacement for the ESN method.

The number of taps q and input dimension d are also relevant for the quadratic $\mathbf{g}_n$ described in Eq. (4.18). The output dimension of this function, $qd + qd(qd + 1)/2$, is also one of the dimensions of the output matrix W_{out} . As this dimension grows, so does the time required to perform ridge regression. This quadratic dependence on q also emphasizes the need to keep the number of taps low.

On top of this, a quadratic output may not be enough. If a quadratic NVAR is trained to do forecasting on a system with total inversion symmetry $\mathbf{u}(t) \rightarrow -\mathbf{u}(t)$, the quadratic terms in the output cannot meaningfully contribute. This reduces the NVAR to a linear VAR, with all the forecasting problems that entails. To use an NVAR on such a system, the nonlinear function $\mathbf{g}_n$ must be expanded to include higher-order terms. This can be done by constructing an ESN and calculating the equivalent NVAR form, but it can also be done more directly by simply adding higher-order monomial terms to $\mathbf{g}_n$. I explore this in Chapter 5. However, this further emphasizes the requirement for a small number of taps, as the number of terms of order n grows as $(qd)^n$.

If these challenges can be overcome, NVARs are an extremely appealing replacement for the ESN approach. ESNs are complicated random networks with many poorly-understood meta-parameters. In contrast, NVARs have only four total parameters, listed in Table 4.1. In addition to being a smaller set of parameters, these parameters are simply much more interpretable. It is not straightforward to imagine what changing the input connectivity of an ESN will do to performance, but changing the taps τ_i has a direct effect on what information is available to the NVAR.

4.5 Summary

VARs and NVARs are, like RCs, a method of transforming an input signal into an output signal. They operate on linear combinations of time-delay taps of the input. In many ways, their construction and use mirrors that of an RC, except that there is no internal dynamic reservoir system. This sidesteps a whole host of questions and decisions relating to reservoir construction, and dramatically simplifies the implementation of VARs and NVARs. However, despite this simplicity, they are now known to be mathematically equivalent to an output layer nonlinear ESN.

NVARs are an existing method, but they are not widely used or discussed. This equivalence opens the door to cross-talk between the NVAR and RC communities. On the RC side, researchers gain a dramatically simpler implementation that is known to be equivalent, and also gain a chance to apply lessons learned in the RC world to the NVAR method. The NVAR approach benefits from the use of ridge regression, long used in the RC field, instead of other linear fitting methods. It also benefits from improvements to evaluation, such as $\tilde{\epsilon}$, and opens the door to using NVARs on traditional RC problems. However, some questions still remain – most notably, how many time-delay taps are practically necessary, and how complicated a nonlinear function will most tasks need?

If these questions are resolved, the NVAR approach is an extremely attractive alternative to the ESN approach. NVARs do not rely on random construction, are simple to implement, and have only a few parameters with direct interpretations. If NVARs work well, it becomes

difficult to recommend ESNs and the complications they entail.

In the next chapter, I apply the NVAR method to the traditional RC tasks of system state inference and chaotic system forecasting. In the process, I resolve these remaining questions, and demonstrate that NVARs are a practical and very simple replacement for traditional ESN approaches.

Chapter 5

NVARs IN PRACTICE

In this chapter, I resolve the practical questions raised in Chapter 4, and demonstrate that NVARs are capable of solving traditional RC problems well with very few taps, simple nonlinearities, and very little training data. Using the example systems from Appendix A, I use NVARs to perform state inference and forecasting on the Lorenz '63 system, forecasting on the double-scroll circuit, and forecasting on the Mackey-Glass time-delay system.

The work presented in this chapter is the result of a collaboration with Daniel J. Gauthier, Erik Bollt, and Wendson A.S. Barbosa [19]. My contribution involves the initial implementation and exploration of the NVAR method, discussion during the development of the Lorenz '63 results, as well as implementing the Lorenz return map, double-scroll forecasting, and Mackey-Glass forecasting.

5.1 State Inference with Lorenz '63

For this inference task, I provide an NVAR with the x and y components of the Lorenz '63 system, and train it to infer the third z variable. Inference tasks like this are useful in situations where some variables of a system are easily measurable, while the rest are only measurable with great care or expense. For example, a mechanical engine can be outfitted at great expense with a large array of internal temperature and pressure sensors, and an NVAR can be trained to infer these variables from easily-accessible external sensors. Once trained, the NVAR can now be reused on engines manufactured without the expensive internal instrumentation.

To build an NVAR, I must choose the time-step Δt , the tap delays τ_i , the nonlinearity $\mathbf{g}_n$, and ridge parameter α . For this task, I use an integration time-step of $\Delta t = 0.05$. Takens' embedding theorem [40] provides guidance on the expected number of taps: as the dimension of the Lorenz attractor is slightly larger than 2, I choose to use 4 equally spaced taps $\tau_0 = 0$, $\tau_1 = 5$, $\tau_2 = 10$, and $\tau_3 = 15$. These taps are spaced to equally cover roughly one Lyapunov period, with the last tap at time $\tau_3 \Delta t = 0.75$.

For the nonlinearity $\mathbf{g}_n$, I use a slightly modified version of the quadratic output introduced in Eq. (4.18). This $\mathbf{g}_n$ includes a constant term, all linear terms of the tap vector $\mathbf{v}$, and all quadratic terms of the tap vector $\mathbf{v}$. That is,

$$\mathbf{g}_n(\mathbf{v}) = 1 \oplus \mathbf{v} \oplus [\mathbf{v} \otimes \mathbf{v}]. \quad (5.1)$$

The addition of a constant term allows the trained linear W_{out} to fit a constant offset, which is useful here as the z component is always positive, even when both x and y are zero. For this task, with $q = 4$ taps and $d = 2$ input dimensions, x and y , the output of $\mathbf{g}_n$ has $1 + 8 + 36 = 45$ components, and the linear output matrix W_{out} has dimension 1×45 .

I train this NVAR on 10 different examples of the Lorenz '63 system, using x and y as the example input and z as the example output. Each training example is 20 time units (400 data points). I then evaluate the NVAR for 45 time units (900 data points) to produce a NRMSE ϵ as in Eq. (2.12), and then average these errors over all 10 trials. During evaluation, the NVAR has access to only x and z , and must produce an estimate of z . I then adjust α manually to minimize this error. This whole process, training the NVAR 10 different times and evaluating the error, takes only a few seconds on a modern computer even with a poorly-optimized implementation.

I find this NVAR has a NRMSE of $1.75 \pm 0.3 \times 10^{-2}$ on this inference task, using the ridge parameter $\alpha = 0.05$. An example of the NVAR during a single training and testing trial is shown in Fig. 5.1. The trained NVAR shows good agreement in the z component during both the training phase (c) and the testing phase (f); in fact, the agreement is so good that when the NVAR signal is drawn on top of the true Lorenz signal, the true signal is not visible underneath.

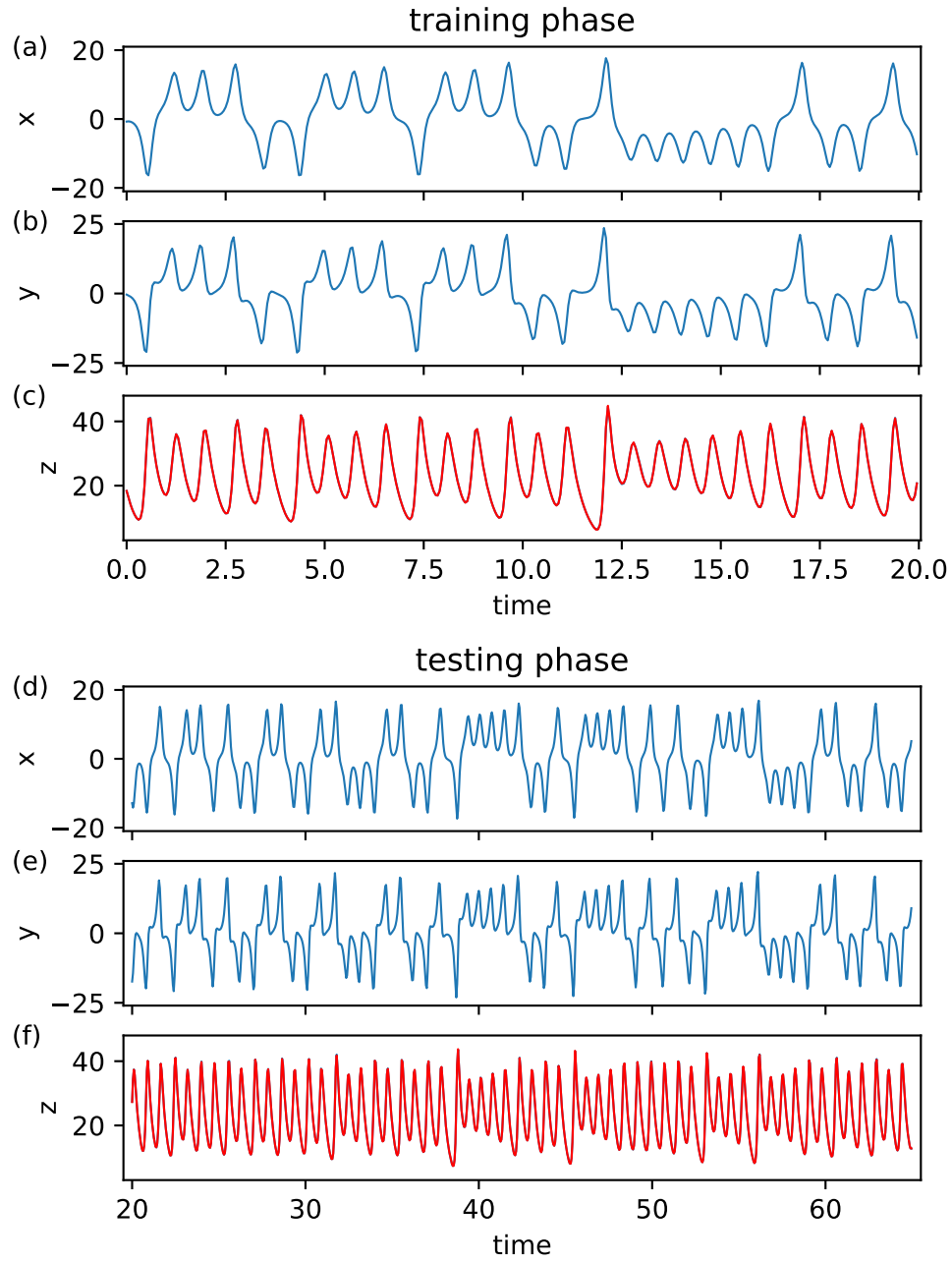


Figure 5.1: (a) – (c) The Lorenz '63 system (blue) during the NVAR training. After the NVAR is trained, it is re-used on the training signal to produce the training output (c, red). (d) – (f) The Lorenz '63 system during NVAR testing, with the NVAR output. In both cases, the NVAR output (red) lies directly on top of the true Lorenz signal (blue).

5.1.1 Output Weights

The output weights W_{out} from a single trial, visualized in Fig. 5.2, have two notable features. First, the largest trained weight by far corresponds to the 1 term in $\mathbf{g}_n$. This accounts for the fact that z is always positive, and justifies the inclusion of the constant term in $\mathbf{g}_n$.

Second, the weights of all the linear terms in x and y are all near zero. This reflects the $(x, y) \rightarrow (-x, -y)$ symmetry of the Lorenz system. Since the NVAR input $\mathbf{u}(t)$ is just these x and y terms, in order to respect this symmetry the NVAR must predict the same $z(t)$ for both $\mathbf{u}(t)$ and $-\mathbf{u}(t)$. This is only possible if the linear terms in x and y have zero coefficients.

In general, an NVAR does not need to respect any symmetries not present in the training input. Specifically, if the NVAR is trained on only a small part of the input system’s phase space, it does not need to respect symmetries outside that space to perform well within that space. Here, however, the NVAR training input $\mathbf{u}_{\text{train}}(t)$ overlaps with its symmetric counterpart $-\mathbf{u}_{\text{train}}(t)$, and so learning the example input necessarily means learning the symmetry.

Later, in Section 5.4, I will investigate a task where the example input does not overlap with its symmetric counterpart, and demonstrate that the NVAR will learn weights that do not agree with this symmetry.

5.2 Forecasting Lorenz ’63

For this task, I use an NVAR in forecasting mode to do system forecasting for the Lorenz ’63 system. That is, I train the NVAR to use all three state variables x , y , and z to perform next-step-ahead prediction of the Lorenz system in Eq. (A.1). Unlike for the inference task, the NVAR has no external driving input and runs autonomously once it is in forecast mode. In analogy to other discrete-time integration algorithms, the choice of time step Δt has a stronger effect on forecasting than inference. Too small a time step is wasted effort, and too large risks inaccuracy. Here, I use a time step of $\Delta t = 0.025$, which results in about 40 steps per Lyapunov period.

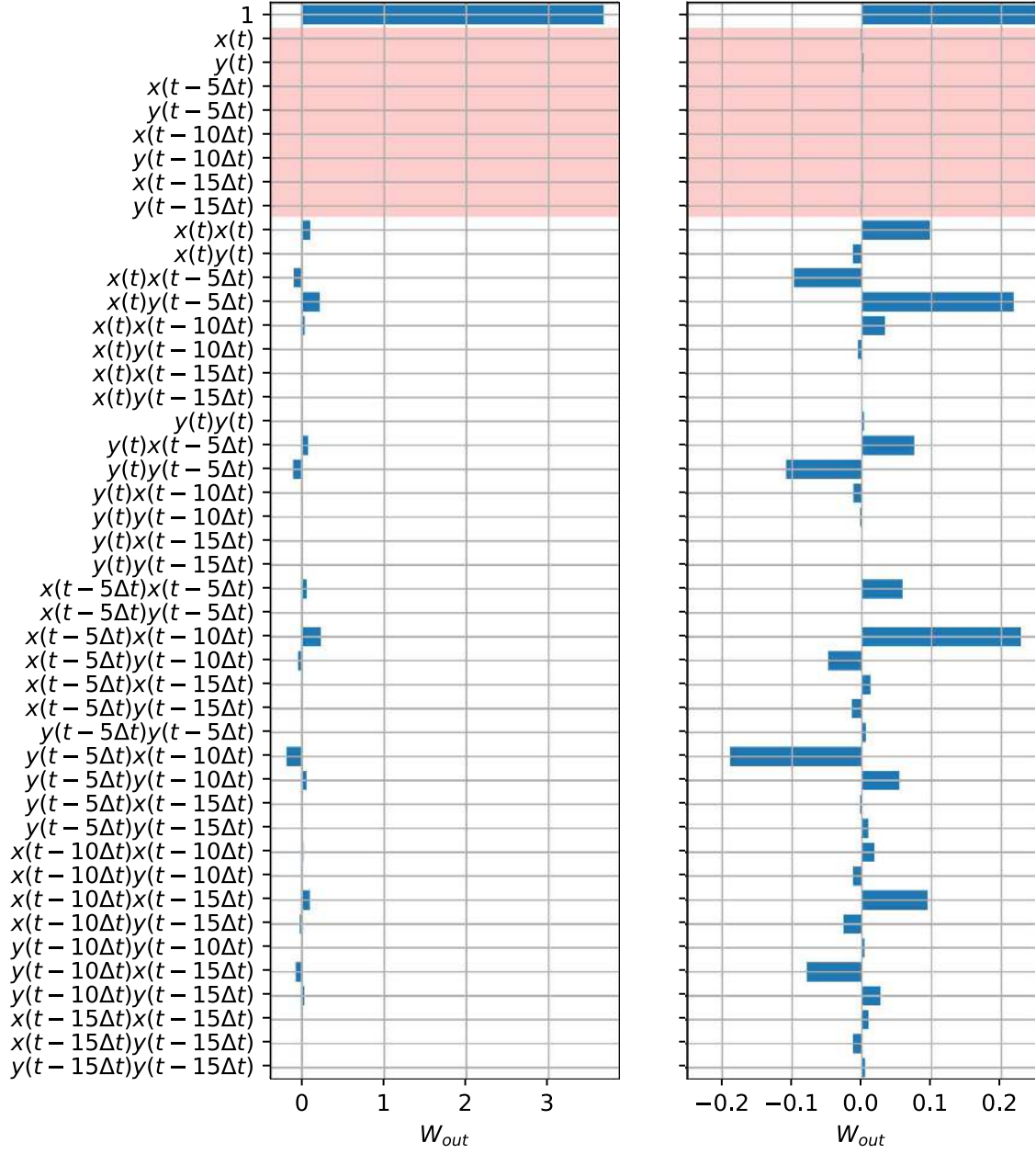


Figure 5.2: Weights in the matrix W_{out} (left) for the NVAR trained for Lorenz system forecasting. On the right, a magnified view of the same. The largest component is the constant offset, to account for the fact that the z component of the Lorenz system is positive, even when both x and y are zero. Also note that the linear terms in x and y (highlighted in red) have weights near zero. As the Lorenz system has a $(x, y) \rightarrow (-x, -y)$ symmetry, the NVAR must have zero weights attached to these terms to respect that symmetry.

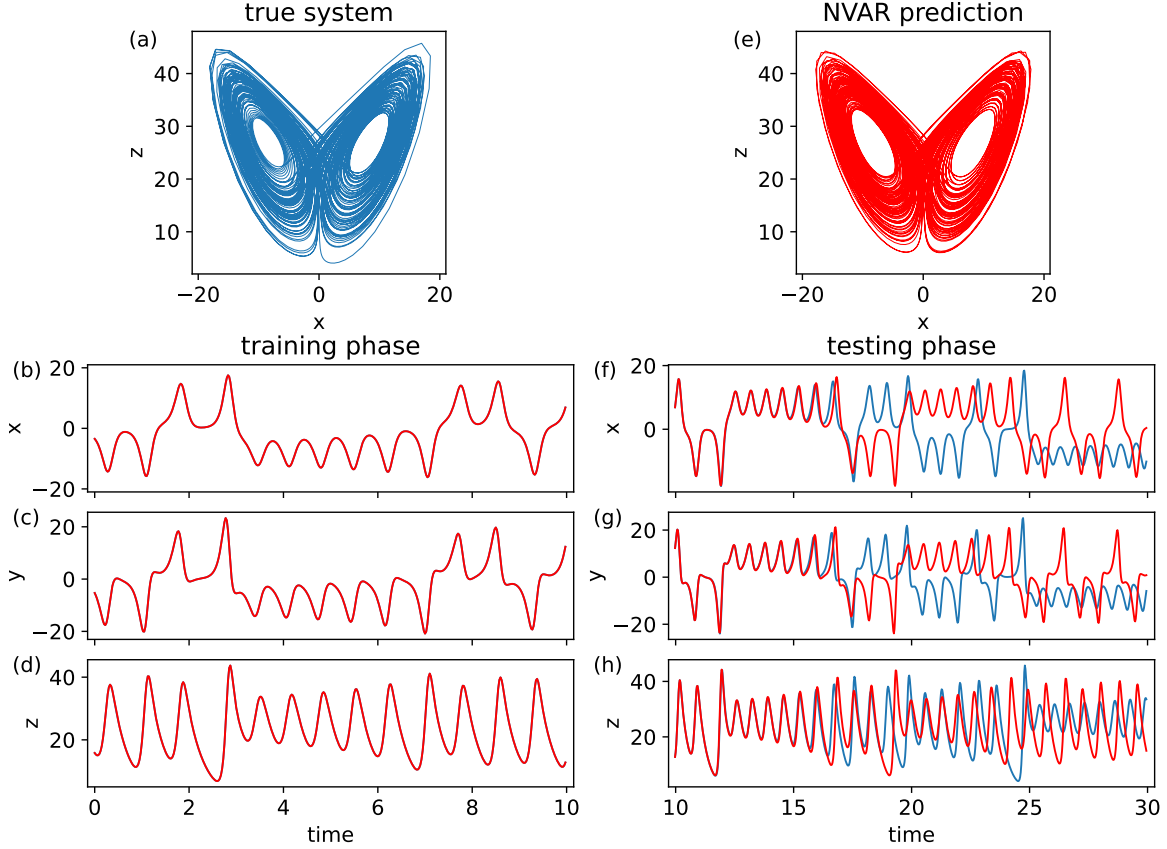


Figure 5.3: The true Lorenz attractor (a) and NVAR predicted attractor (e) for a single training trial. (b) – (d) True Lorenz system (blue) during training overlaid with NVAR output (red) calculated after training is complete. (f) – (h) True (blue) and NVAR forecasted output (red). The NVAR shows good agreement with the true system as far as 6 Lyapunov periods in to the autonomous forecast.

For this forecasting task, I use only 2 taps, $\tau_0 = 0$ and $\tau_1 = 1$. That is, the NVAR has access to the two previous time steps, and I find this is enough for the forecasting task. I use the quadratic nonlinearity with constant term described in Eq. (5.1). With $q = 2$ taps and $d = 3$ input dimensions, the nonlinear output of $\mathbf{g}_n$ has $1 + 6 + 21 = 28$ components, and so W_{out} has dimension 3×28 .

Again, I train this NVAR on 10 different examples of the Lorenz '63 system, each time training on 10 time units (400 data points). I then evaluate the NVAR by performing an autonomous forecast for one Lyapunov period to calculate a NRMSE ϵ_1 , as in Eq. (2.12), and then average this over the 10 trials to produce $\tilde{\epsilon}$ in Eq. (2.13).

The performance of this NVAR during a single training trial is shown in Fig. 5.3. A long autonomous forecast from the NVAR produces an attractor (e) that shows qualitative agreement with the true Lorenz attractor (a), and produces a NRMSE over one Lyapunov period of $4.51 \pm 0.85 \times 10^{-3}$ using $\alpha = 2.5 \times 10^{-6}$, about five times better than the ESNs described in Table 3.2. This NVAR performs better than the ESNs despite the fact that it is simpler to describe and implement, does not require lengthy optimization, and trains at least an order of magnitude faster. The training data used for the NVAR is also very light: only 10 time units, corresponding to 400 points at this time step, is enough to train the NVAR. During training, the NVAR’s output (b) – (d) matches the true signal so well it obscures it, and during testing, the autonomous forecast (f) – (h) only visibly diverges from the truth after 6 Lyapunov exponents.

5.2.1 z Return Map

The z variable of the Lorenz ’63 system has a functional relation between successive local maxima. This is demonstrated visually by finding the local maxima z_i of z , and then plotting z_i with respect to z_{i+1} [41]. This *return map* neatly summarizes the long-term behavior of the z variable, and comparing two such maps provides a quick way to qualitatively compare two systems. This comparison has been used previously to verify that a trained RC can replicate the Lorenz ’63 dynamics [11].

In many ways, this comparison is similar to the visual inspection of the reproduced attractor in Fig. 5.3. However, the return map is a simpler shape and is much more sensitive to error than the attractor itself, even when comparing two maps by eye. Indeed, the return map is so sensitive that it depends on the specific integrator used to find the true Lorenz ’63 solution. In Fig. 5.4, I show this return map calculated on a Lorenz solution produced by four different integrators. All four methods agree on a large scale, but the zoomed view in (b) reveals their differences. Using a relative tolerance of 10^{-3} , the Runge-Kutta 3(2) method produces a narrow, nearly one-dimensional return map slightly offset from the much wider return map produced by Runge-Kutta 5(4). Both algorithms agree after sharpening the tolerance to 10^{-5} . In this chapter, I use the Runge-Kutta 3(2) [34] integrator with 10^{-3}

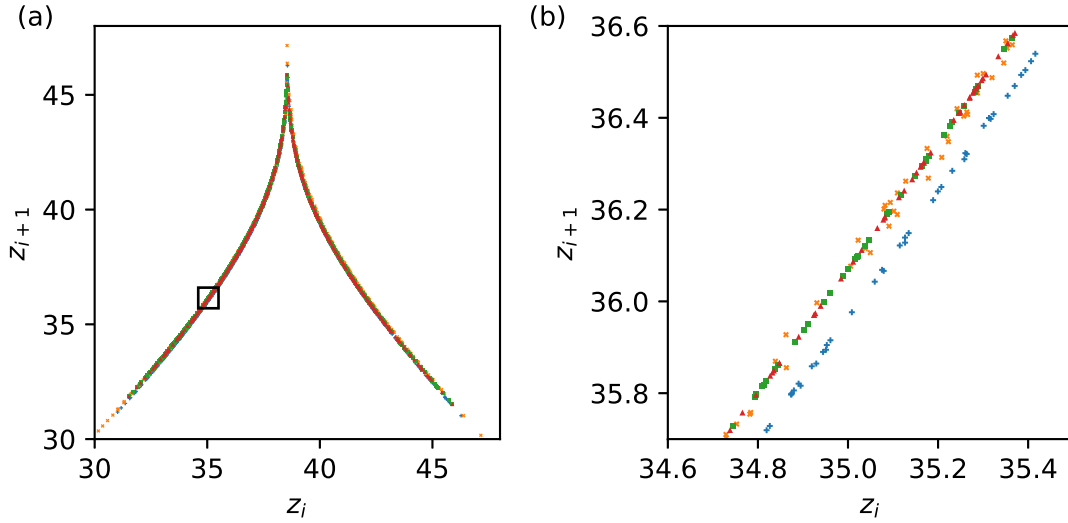


Figure 5.4: (a) The z return map of the Lorenz '63 system, integrated using various methods. (b) Detail of the region marked in (a). Using a relative integration tolerance of 10^{-3} , the Runge-Kutta 3(2) (blue +) differs from R-K 5(4) (orange $\times$) in both position and spread. Both methods converge after sharpening the tolerance to 10^{-5} (green square and red triangle, respectively).

relative tolerance for both the Lorenz '63 and double-scroll systems.

The values of the maxima in the discrete-time solutions for both the NVAR and Lorenz '63 system depend on the time step Δt used for integration, as the true maximum may be achieved in between the discrete time steps. To better reproduce the true Lorenz '63 return map, and to reduce the effect Δt has on the NVAR return map, I interpolate the z solutions by using a degree-4 spline. The local maxima are then found on this interpolated spline [42].

Figure 5.5 shows the return maps for both the true Lorenz system and the NVAR. Qualitatively, there is good agreement between the two. The NVAR return map almost completely obscures the true Lorenz return map in (a). Upon close inspection in (b), I see that the NVAR return map has a width that is wider than that of the true return map. This can be improved by extending the training time of the NVAR. The difference between the two return maps is already small even when the NVAR is trained for only 10 time units; when training for 20 time units, as in Fig. 5.6, the NVAR's predicted return map narrows

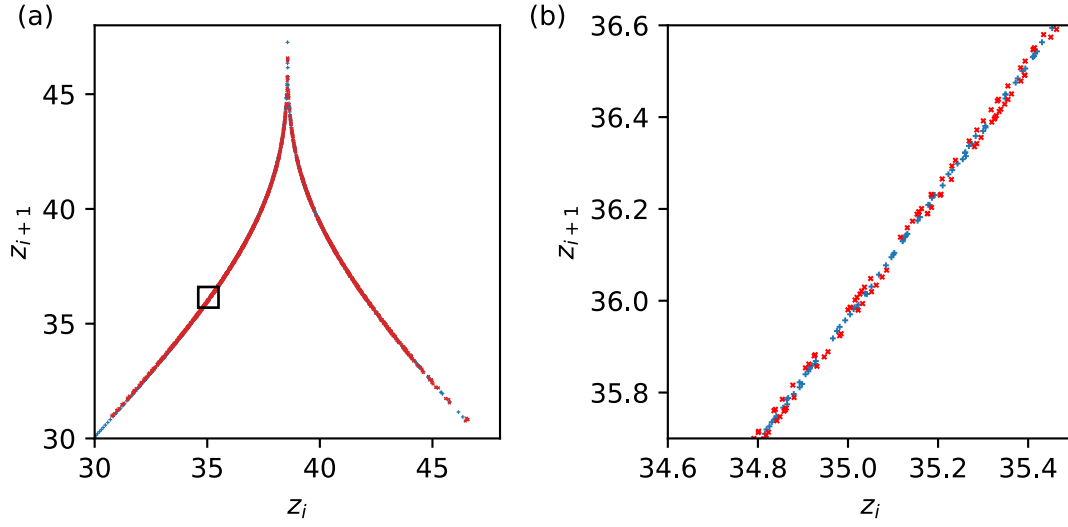


Figure 5.5: (a) The z return map of the Lorenz '63 system (blue +) overlaid with the z return map of the NVAR forecast (red x), trained for 10 time units. The NVAR reproduces the long-term dynamics of the z variable accurately enough at this scale that it is difficult to see the true return map underneath. (b) Detail of the region marked in (a). At this scale, it is possible to see that the NVAR return map is wider than the true Lorenz return map, although they lie on top of each other. This reproduction can be improved by training the NVAR for longer (see Fig. 5.6).

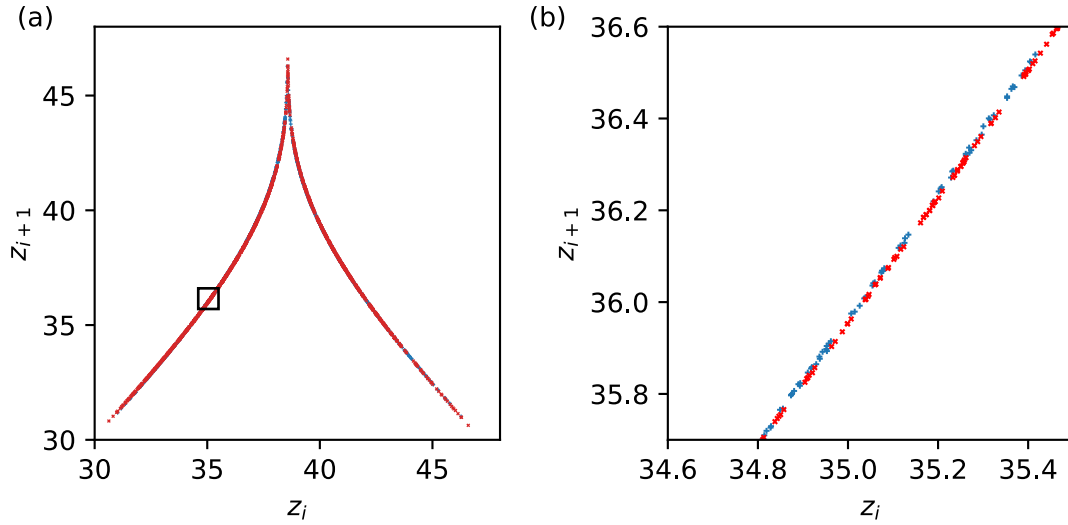


Figure 5.6: (a) The z return map of the Lorenz '63 system (blue +) overlaid with the z return map of the NVAR forecast (red x), trained for 20 time units. (b) Detail of the region marked in (a). With more training time, the NVAR return map is narrower and more accurately reproduces the Lorenz return map.

and lies on top of the true return map.

As with the attractor comparison, a quantitative measure of return map similarity would be a strong candidate for replacing the flawed NRMSE measure.

5.2.2 Fixed Points

For a more quantitative measure of how well the NVAR learns the dynamics of the Lorenz system, I calculate the fixed points of the trained NVAR and then find their distance to the true Lorenz steady states. This comparison has also been used for reservoir computers [43]. For simplicity and consistency, in this dissertation I will refer to both the discrete-time fixed points of the NVAR and the continuous-time steady states of Lorenz and other systems as “fixed points”.

By setting the derivatives in Eq. (A.1) to zero, I find three fixed points of the Lorenz system located at

$$[x, y, z] = [0, 0, 0], \quad (5.2)$$

$$= [\pm\sqrt{\beta(\rho-1)}, \pm\sqrt{\beta(\rho-1)}, (\rho-1)]. \quad (5.3)$$

To find the fixed points of the NVAR system, I perform a trial single-step forecast with a constant past history, and use the Powell hybrid method root-finding algorithm [44, 45] to find the specific values of past history that result in an unchanged forecast. This root-finding algorithm is used three times, using the three true Lorenz fixed points as initial guesses.

To allow easy comparison of the accuracy of these fixed points with other systems, I calculate the distance between this fixed points in a uniformly scaled space where the Lorenz63 system has unit variance. For this trained NVAR, the distance from the zero fixed point is $1.2 \pm 1.4 \times 10^{-3}$, and the distances between the predicted and true positive and negative fixed points are $11.6 \pm 3.0 \times 10^{-4}$ and $7.4 \pm 1.5 \times 10^{-4}$, respectively.

The NVAR shows good agreement on the zero fixed point, as the variance caused by different training windows overlaps with the true location. The other two fixed points are consistently offset from the truth. However, in all three cases, the fixed points lie within

approximately 0.1% of the true location, despite the fact that the training data does not include any of these points.

The NVAR also occasionally exhibits fixed points that do not have a corresponding fixed point in the Lorenz system. These spurious fixed points depend on which specific segment of Lorenz data is used to train the NVAR, and will move around or disappear as the training data changes. Future research could identify how many of these spurious fixed points exist, as well as how to prevent them or otherwise control for their presence.

5.2.3 Output Weights

The particular choice of nonlinearity $\mathbf{g}_n$ for this task completely includes all terms in the vector field of the Lorenz system in Eq. (A.1). That is, the Lorenz system is described by a polynomial of degree 2, and the choice of $\mathbf{g}_n$ allows the NVAR to describe *any* polynomial of degree 2. This raises a legitimate question: is the NVAR simply performing so well because it was equipped ahead of time with the perfect tools?

The answer in a general context is no. Just as a finite truncation of an infinite Taylor series can approximate non-polynomial functions arbitrarily well, a continuous nonlinear dynamical system can be approximated arbitrarily well by a finite Volterra series [46], and NVARs with polynomial outputs are finite Volterra series with very simple kernels. From this fact, I expect NVARs with quadratic nonlinearity to be able to approximate even non-polynomial systems, and so it is not surprising that it can also approximate quadratic polynomial systems.

Another way to allay these concerns is to look at other non-polynomial systems, as in the next few sections. However, even with the Lorenz system, I see that this is not the case by inspecting the trained weight matrix W_{out} in Fig. 5.7. The terms on the right hand side of the Lorenz system (marked in red) are present with non-zero weights in the output matrix, but there are also significant contributions from other terms, and there does not appear to be a preference for the NVAR to rely on Lorenz terms only.

From this I derive an interpretation: a trained NVAR is finding the best possible one-step-ahead integrator it can find, given the example input and the available terms in $\mathbf{g}_n$.

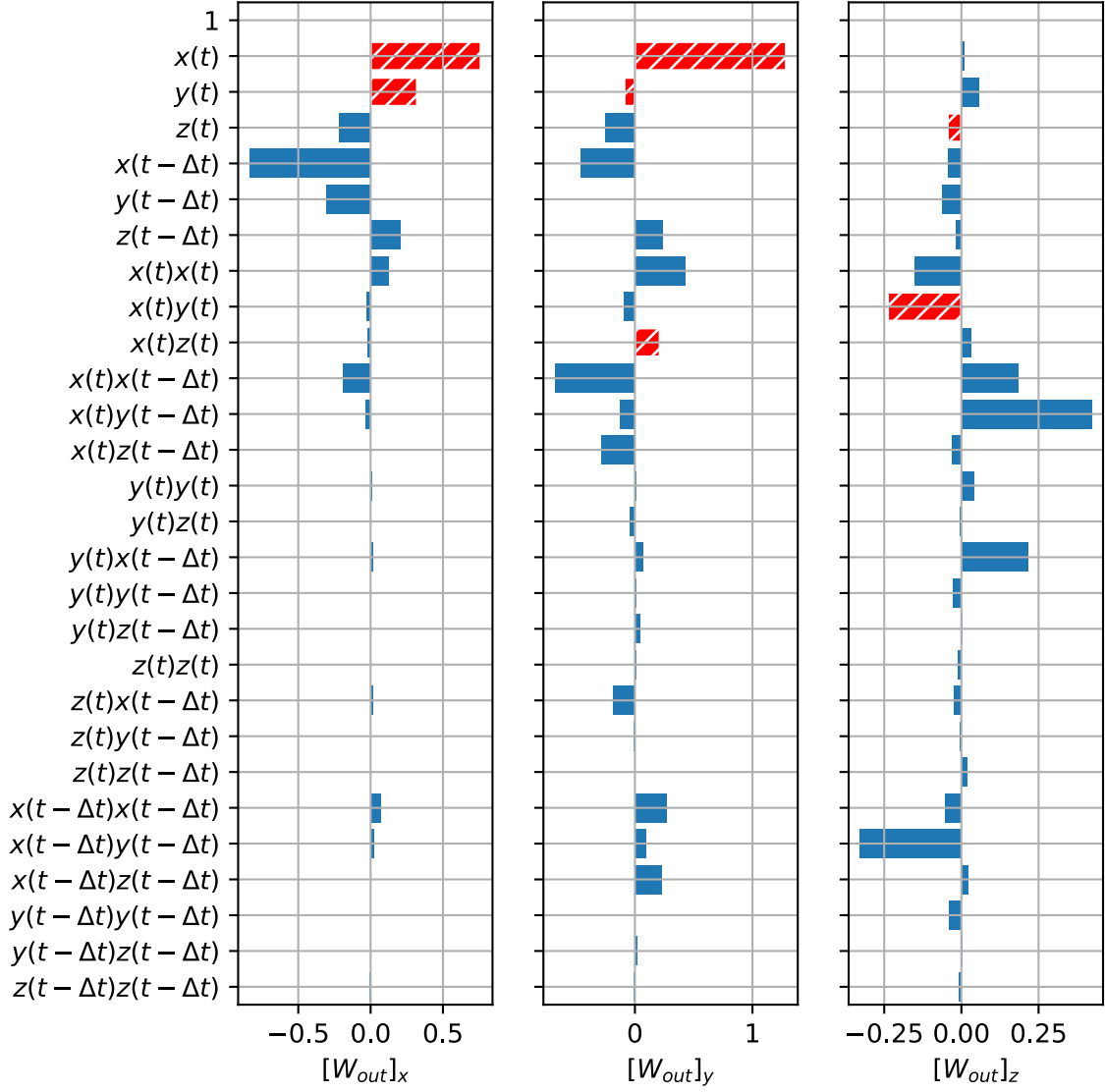


Figure 5.7: Weights in the matrix W_{out} for the NVAR trained for Lorenz system forecasting. Terms that appear directly in the Lorenz system (Eq. (A.1)) are in dashed red. There does not appear to be any correlation between a large weight in the NVAR output, and presence in the Lorenz equations.

When the time step is large enough, a prediction that is linear in the vector field becomes more and more inaccurate, and so the NVAR relies on those terms not present in the vector field to make up the difference. In other words, the NVAR is learning how to move from $\mathbf{u}(t)$ to $\mathbf{u}(t + \Delta t)$, and for coarse time steps Δt this is not the same as learning the vector field of $\mathbf{u}(t)$.

5.2.4 Computational Complexity

To compare the computational speed of training an NVAR on the Lorenz forecasting task against that of training an ESN, I estimate the number of multiplications required for training in each method. To do this, I first identify the parameters in each method that contribute most to the computational complexity.

Both the NVAR and ESN approaches share three parameters: the number of warm-up steps M_{warmup} , the number of training steps M_{train} , and the size of the feature vector N_{total} . For an NVAR, N_{total} is the dimension of $\mathbf{g}_n(\mathbf{v})$, while for an ESN it is the dimension of $\mathbf{g}(\mathbf{r})$. ESNs also have the number of nodes in the network N , as well as a parameter σ_r that controls the sparsity of internal network connections.

For ESNs, I conservatively assume that sparse matrix multiplication has no additional overhead, and that special function evaluations (such as $\tanh$) are free. In reality, such evaluations may involve many multiplications, and so ESNs will be more costly than my estimate expects.

The largest contribution to the computational complexity for the NVAR is the ridge regression performed over the training time, which is $\mathcal{O}(M_{\text{train}} N_{\text{total}}^2)$. The largest contributions for an ESN are the matrix multiplication $W_r \mathbf{r}$ over the warm-up and training time, which is $\mathcal{O}(\sigma_r (M_{\text{warmup}} + M_{\text{train}}) N^2)$, and the ridge regression over the training time, which is $\mathcal{O}(M_{\text{train}} N_{\text{total}}^2)$.

I estimate the speed-up by summing the largest contributions for the ESN and then divide by the contributions from the NVAR. I perform this comparison against the ESNs used in Chapter 3 [18], as well as ESNs found in other work [11, 23]. These comparisons are summarized in Table 5.1. Even with this conservative analysis, the NVAR achieves a

Method	M_{warmup}	M_{train}	N_{total}	N	σ_r	speed-up
NVAR	2	400	28	-	-	-
ESN [18]	1000	1000	100	100	0.01–0.05	33–163
ESN [11]	? (set to 0)	5000	300	300	0.02	1.5×10^3
ESN [23]	10^5	6×10^4	4000	2000	0.02	3.2×10^6

Table 5.1: Estimated speed-up of the NVAR on the Lorenz ’63 forecasting task, compared to ESNs described in Chapter 3 [18] and in other work [11, 23].

significant speed-up over the ESNs while retaining high accuracy.

5.2.5 Robustness to Noise

To explore how robust the NVAR approach is to noise, I train this same NVAR construction on the Lorenz ’63 system modified to contain additive Gaussian noise on the right-hand side of the differential equations for each variable. I set the scale of this noise to be 12% of the root-mean-square values of each variable, which are 7.9, 9.0, and 8.6 for x , y , and z respectively.

To generalize well after training, I increase the ridge parameter $\alpha = 1.4 \times 10^{-2}$, but otherwise leave the NVAR unmodified. Once the NVAR is trained on the noisy data, I then perform a forecast with the NVAR and compare the forecast to the noise-free Lorenz ’63 dynamics, and find a one Lyapunov period error of $\epsilon_1 = 4.18 \times 10^{-2}$. The performance of this trained NVAR is summarized in Fig. 5.8. This low error indicates that the NVAR is still capable of learning the Lorenz ’63 system even when it is trained on noisy data, and this is supported by how well the NVAR is able to reproduce the Lorenz attractor.

5.3 Forecasting the Double-Scroll Circuit

For this task, I use an NVAR in forecasting mode to use all three state variables V_1 , V_2 , and I of the double-scroll circuit described by Eq. (A.3). I use the same tap delays $\tau_0 = 0$, $\tau_1 = 1$ as the Lorenz case, but I modify the choice of nonlinearity.

Unlike the Lorenz system, the double-scroll circuit cannot be described by a polynomial

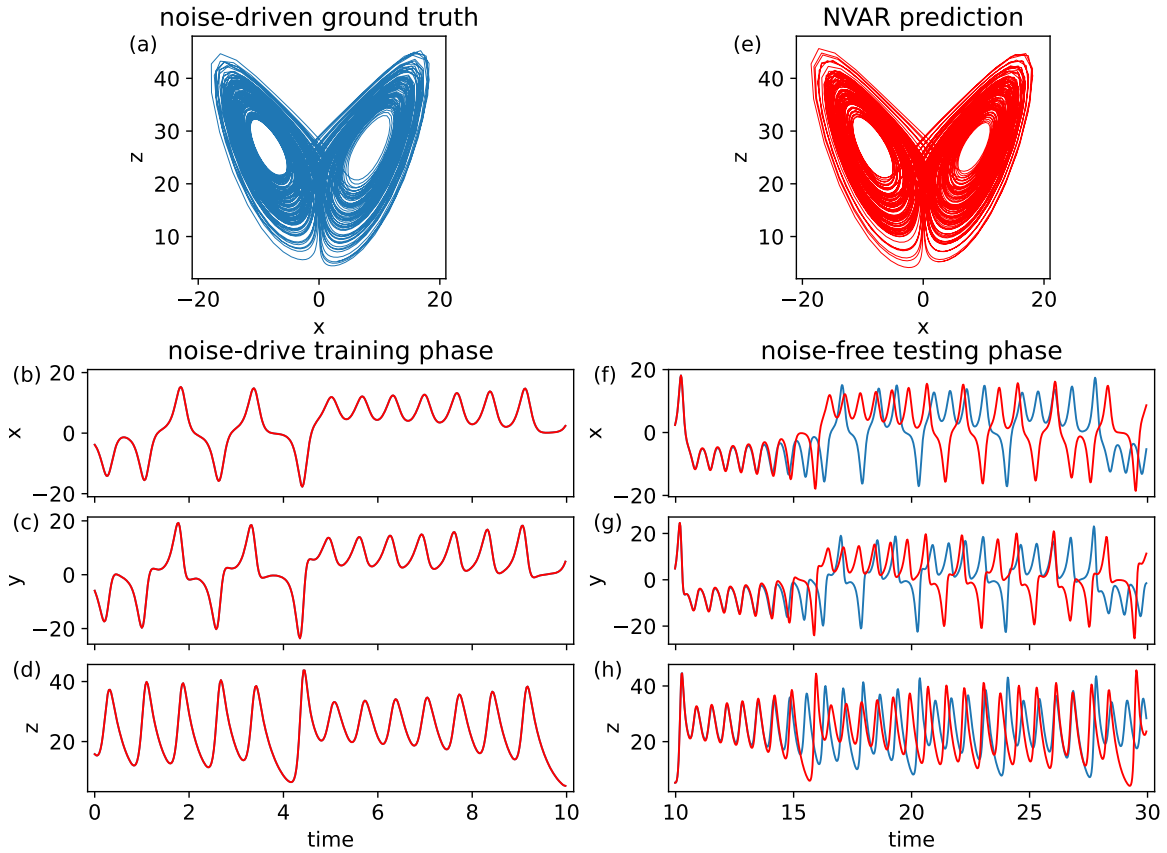


Figure 5.8: The long-term trajectory of a noise-driven Lorenz '63 system (a) and NVAR predicted attractor (e) for a single training trial. (b) – (d) Noise-driven Lorenz system (blue) during training overlaid with NVAR output (red) calculated after training is complete. (f) – (h) True, noise-free Lorenz (blue) and NVAR forecasted output (red).

of degree 2. Indeed, the double-scroll equations are odd, and have full inversion symmetry. On top of this, the double-scroll attractor includes this symmetry; it is identical to its symmetric partner. As discussed in Section 5.1.1, this drives the contribution of any even terms in $\mathbf{g}_n$ to zero in the trained weight matrix W_{out} . In light of this, I use a nonlinear function that includes all linear terms of the tap vector $\mathbf{v}$ as well as all terms with degree 3, but includes no even terms. That is,

$$\mathbf{g}_n(\mathbf{v}) = \mathbf{v} \oplus [\mathbf{v} \otimes \mathbf{v} \otimes \mathbf{v}]. \quad (5.4)$$

With $q = 2$ taps and $d = 3$ input dimensions, the output of $\mathbf{g}_n$ has $6 + 56 = 62$ components, and so W_{out} has dimension 3×62 .

The Lyapunov time of the double-scroll circuit is roughly 10 times longer than that of the Lorenz system. To ensure a fair comparison, I extend the training time of the NVAR from 10 to 100 units to keep the number of Lyapunov times covered by the training similar for both cases. I also set $\Delta t = 0.25$, to ensure that both the Lorenz and double-scroll cases use 400 data points during training.

Other than these modifications, the method for using the NVAR to forecast the dynamics of this system proceeds exactly as for the Lorenz system, training the NVAR on 10 different examples of the double-scroll circuit and then evaluating the NRMSE for one Lyapunov period each. The performance of this NVAR during a single training trial is shown in Fig. 5.9.

Once again, a long autonomous forecast (e) shows the NVAR can reproduce the shape of the double-scroll attractor (a), and over one Lyapunov period has an NRMSE of $2.15 \pm 0.63 \times 10^{-2}$ using $\alpha = 1.0 \times 10^{-3}$, about $1.5\times$ better than the ESNs described in Table 3.3. During training (b) – (d) the NVAR matches the system so well the true signal cannot be seen underneath, and once autonomous forecasting begins in (f) – (h), the NVAR’s prediction lies on top of the true system for 6 Lyapunov periods.

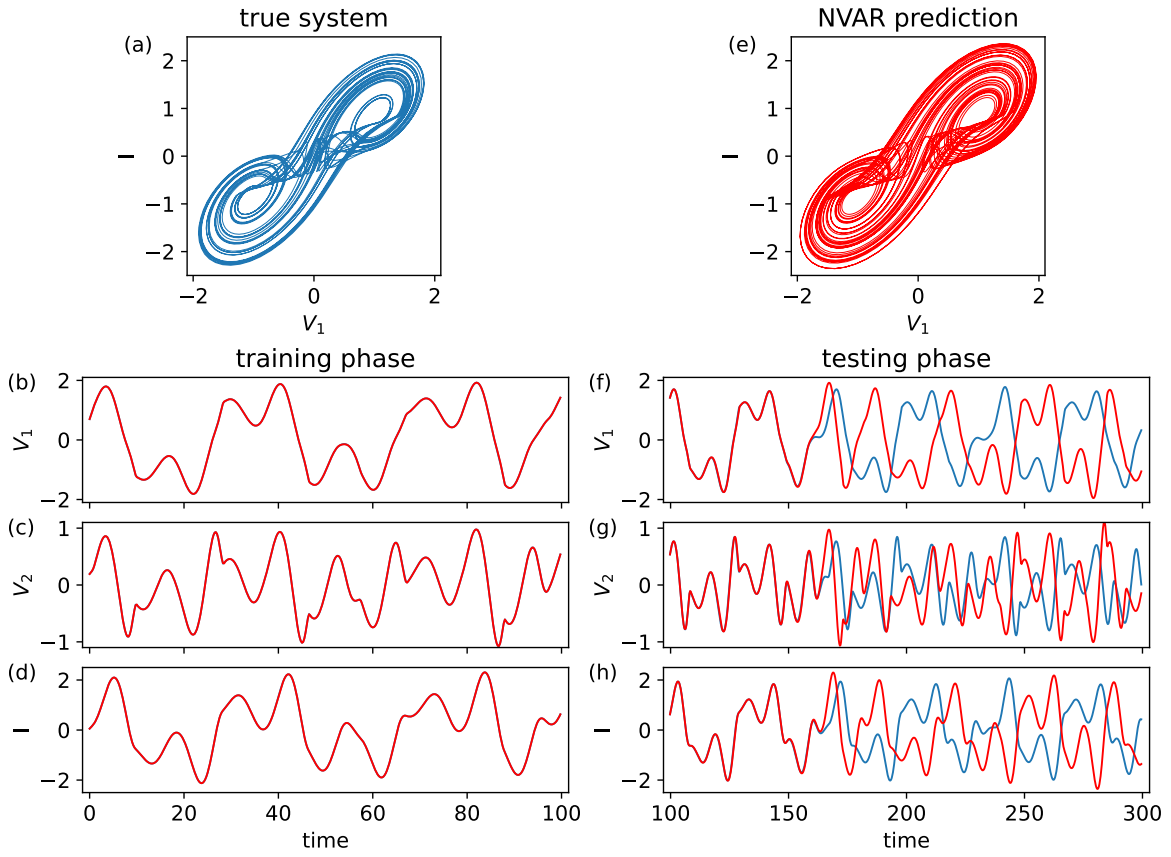


Figure 5.9: The true double-scroll attractor (a) and NVAR predicted attractor (e) for a single training trial. (b) – (d) True double-scroll system (blue) during training overlaid with NVAR output (red) calculated after training is complete. (f) – (h) True (blue) and NVAR forecasted output (red). Here, the NVAR shows good agreement with the true system as far as 6 Lyapunov periods.

5.3.1 Fixed Points

Again, I look for the fixed points of this trained NVAR and find their distance to the true fixed points of the double-scroll circuit, in a space uniformly scaled so that the double-scroll circuit has unit variance. Setting the derivatives in Eq. (A.3) to 0 and solving yields the transcendental equation

$$0 = \frac{V_1}{R_2}(R_1 - R_2 - R_4) + 2R_1 I_r \sinh \left(\alpha V_1 \left(1 - \frac{R_4}{R_1} \right) \right). \quad (5.5)$$

For the parameters I use, this yields three solutions for V_1 . If V_1^+ is the positive solution, this corresponds to three fixed points at

$$[V_1, V_2, I] = [0, 0, 0], \quad (5.6)$$

$$= [V_1^+, \pm V_1^+ R_4 / R_1, \pm V_1^+ / R_1]. \quad (5.7)$$

Since my choice of $\mathbf{g}_n$ has only odd polynomial powers and no constant term, it is symmetric about the origin and predicts the zero fixed point exactly. The distance from the true positive fixed point is $2.7 \pm 0.3 \times 10^{-3}$. Due to the inversion symmetry, the negative fixed point is precisely the same distance. Although the variance from repeated trials does not overlap the true fixed points, they do lie within 0.3% of the truth.

This NVAR performs well at short-term forecasting and fixed point prediction, despite not having access to the infinite number of polynomial terms in the expansion of the double-scroll equations. This demonstrates a possible practical approach to NVAR design: keep adding delay taps and higher polynomial terms to the nonlinear function $\mathbf{g}_n$ until performance is acceptable. However, for NVARs that require many taps or operate on high-dimensional inputs, this very quickly leads to an explosion in the number of weights trained in W_{out} , and consequently a huge increase in training time. In the next section, I explore how careful use of tap spacing can keep the number of taps required low.

5.4 Forecasting Mackey-Glass

For this task, I again use an NVAR in forecasting mode, this time using the single Mackey-Glass state variable u in Eq. (A.4) as input, with $\Delta t = 0.2$. I use the `jitcdde` Python package [47] to integrate this time-delay differential equation. For this system, using two adjacent taps for the NVAR as in the previous examples is not enough, as the Mackey-Glass system behavior depends strongly on both the value of $u(t)$ as well as $u(t - 17)$. One way to access this value in the NVAR is to add taps at every time step until they reach $t = 17$, resulting in 85 total taps. Even with only a quadratic output, this requires training a W_{out} with dimension 1×3741 . This cost can be avoided by increasing the time step Δt , thus reducing the total number of taps, but this solution comes with the cost of a much more granular prediction. Granularity may not be a problem, as up to a point, as the NVAR's predictions can be interpolated to fill in the gaps. However, using a coarser Δt also changes the behavior of the NVAR, and for this task results in poor performance.

Another way to access the value at $u(t - 17)$ is to use sparse taps. Here, I use three clusters of two adjacent taps: one cluster at $\tau_0 = 0$ and $\tau_1 = 1$, one cluster at $\tau_2 = \lfloor 17/2\Delta t \rfloor$ and $\tau_3 = \tau_2 + 1$, and one cluster at $\tau_4 = \lfloor 17/\Delta t \rfloor$ and $\tau_5 = \tau_4 + 1$. Here, $\lfloor a \rfloor$ refers to the integer floor of a . This gives the NVAR access to information at $u(t - 17)$ as well the midpoint $u(t - 17/2)$. I find that adding the tap cluster at the midpoint improves performance by a factor of 3, and increases the number of features available to W_{out} to be comparable to the other tasks in this chapter.

The Mackey-Glass system exhibits full inversion symmetry. However, unlike the double-scroll circuit, the Mackey-Glass system has two distinct attractors that do not overlap. Any solution on the positive attractor will remain positive, and never cross over zero to the negative attractor. Since I am only training the NVAR on a positive solution to Mackey-Glass, I include even terms in $\mathbf{g}_n$. In fact, I find that including these even terms improves NRMSE performance. In Section 5.4.2, I demonstrate that the trained NVAR is assigning these terms non-zero weights.

Therefore, I use a nonlinear function $\mathbf{g}_n$ that includes the constant, linear, quadratic,

and cubic terms constructed from the tap vector $\mathbf{v}$,

$$\mathbf{g}_n(\mathbf{v}) = 1 \oplus \mathbf{v} \oplus [\mathbf{v} \otimes \mathbf{v}] \oplus [\mathbf{v} \otimes \mathbf{v} \otimes \mathbf{v}]. \quad (5.8)$$

With $q = 6$ taps and $d = 1$ input dimension, the output of $\mathbf{g}_n$ has $1 + 6 + 21 + 56 = 84$ components, and so W_{out} has dimension 1×84 .

The Lyapunov time of the Mackey-Glass system is roughly 100 times longer than that of the Lorenz system. However, I find that a relatively fine step size $\Delta t = 0.2$ is required for the NVAR to learn the system, corresponding to 500 points per Lyapunov period. I train this NVAR for 1000 time units, corresponding to 5000 data points total. Though this is many more discrete data points, it covers about 10 Lyapunov times, comparable to the training time used for the other systems.

This NVAR is much more data hungry than the previous examples, requiring more training data to produce an acceptable result. This may be due to the wide tap spacing: values at $u(t - 17)$ are less correlated to $u(t)$, while $u(t - \Delta t)$ is more correlated, and so the NVAR needs more data to see every relevant combination of values at the taps.

Other than these modifications, the methodology for training and evaluating the NVAR is exactly as for the other two systems. The performance of this NVAR during a single training trial is shown in Fig. 5.10. Over one Lyapunov period, this NVAR has a NRMSE of $5.77 \pm 0.42 \times 10^{-2}$ with $\alpha = 9 \times 10^{-3}$.

The long autonomous forecast (c) of the NVAR shares important features with the true attractor (a), most notably the gap between the bands. However, it does not get some fine details correct, notably the shape of the heel in the upper-left corner and the overlapping bands in the middle. This is reflected in the higher NRMSE than the other tasks. However, as before the NVAR prediction completely obscures the true system signal during training (b), and once autonomous forecasting (d) begins the NVAR forecast covers the true system for 2 Lyapunov periods.

The shape of this heel can be better reproduced by including higher-order terms in the nonlinear function $\mathbf{g}_{\text{out}}$. However, it is not possible to do this with monomial terms as I have been doing without dramatically increasing the training time of W_{out} . Adding a

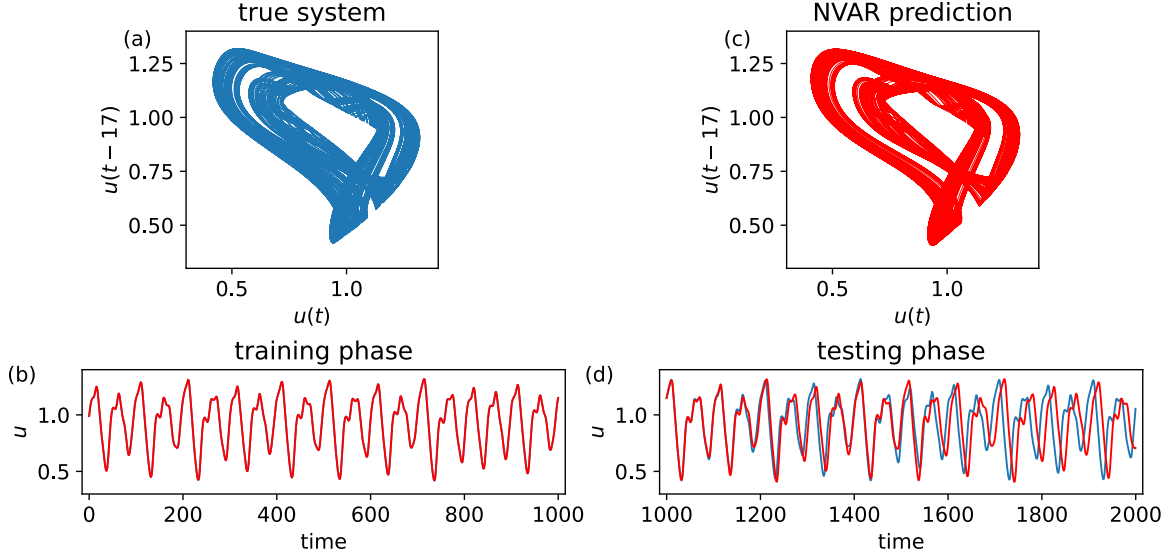


Figure 5.10: The true Mackey-Glass attractor (a) and NVAR predicted attractor (c) for a single training trial. (b) True Mackey-Glass system during training overlaid with NVAR output (red) calculated after training is complete. (d) True (blue) and NVAR forecasted output (red). Here, the NVAR shows good agreement with the true system as far as 2 Lyapunov periods.

fourth-order term to $\mathbf{g}_{\text{out}}$ increases the number of weights in W_{out} by a factor of 3. This problem becomes even worse on input systems with a dimension higher than $d = 1$.

It may be possible to instead add nonlinear terms to $\mathbf{g}_{\text{out}}$ with an infinite power series, such as $e^{v_i v_j}$ or $\sinh v_i \cosh v_j$. This would provide access to high-order nonlinearity without a corresponding large increase in training time. This is one important avenue for future research, and this Mackey-Glass forecasting example demonstrates that cubic terms are not completely sufficient for accurate forecasts. Nonetheless, the cubic NVAR used here shows promising performance from a simple model.

5.4.1 Fixed Points

Setting the derivative in Eq. (A.4) to 0 and assuming a constant past yields three fixed points, located at

$$u = 0, \tag{5.9}$$

$$u = \pm \left(\frac{\beta}{\gamma} - 1 \right)^{1/n}. \tag{5.10}$$

I use these as initial guesses to find the fixed points of the trained NVAR, in a space uniformly scaled so that Mackey-Glass has unit variance.

The distance to the true positive fixed point is $2.25 \pm 0.03 \times 10^{-3}$. The variance from repeated trials does not overlap the true fixed point, however, it does lie within 0.3% of the truth.

The NVAR completely fails to predict the 0 fixed point and the negative fixed point. This is a consequence of my choice not to respect the known symmetries of the Mackey-Glass system in favor of more accurate prediction on the positive attractor. If accuracy on the non-positive fixed points is desired, the NVAR can be either modified to respect that symmetry ahead of time, or trained on data from both the positive and negative attractors.

5.4.2 Output Weights

The output weights for this task are in Fig. 5.11. As expected from the failure to predict the non-positive fixed points, the NVAR assigns non-zero weights to those terms (in red) that break the total inversion symmetry. As the training input does not include any data from the negative Mackey-Glass attractor, the NVAR has no cause to learn this symmetry and instead utilizes these terms to improve forecasting accuracy on the positive attractor. That the NVAR assigns non-zero weights to these terms, and that the resulting NVAR performs better than one constructed without these terms, justifies my inclusion of the even terms in this NVAR and demonstrates that an NVAR can use symmetry-breaking terms productively as long as it is only used for forecasting in an area of the underlying system that does not contain that symmetry.

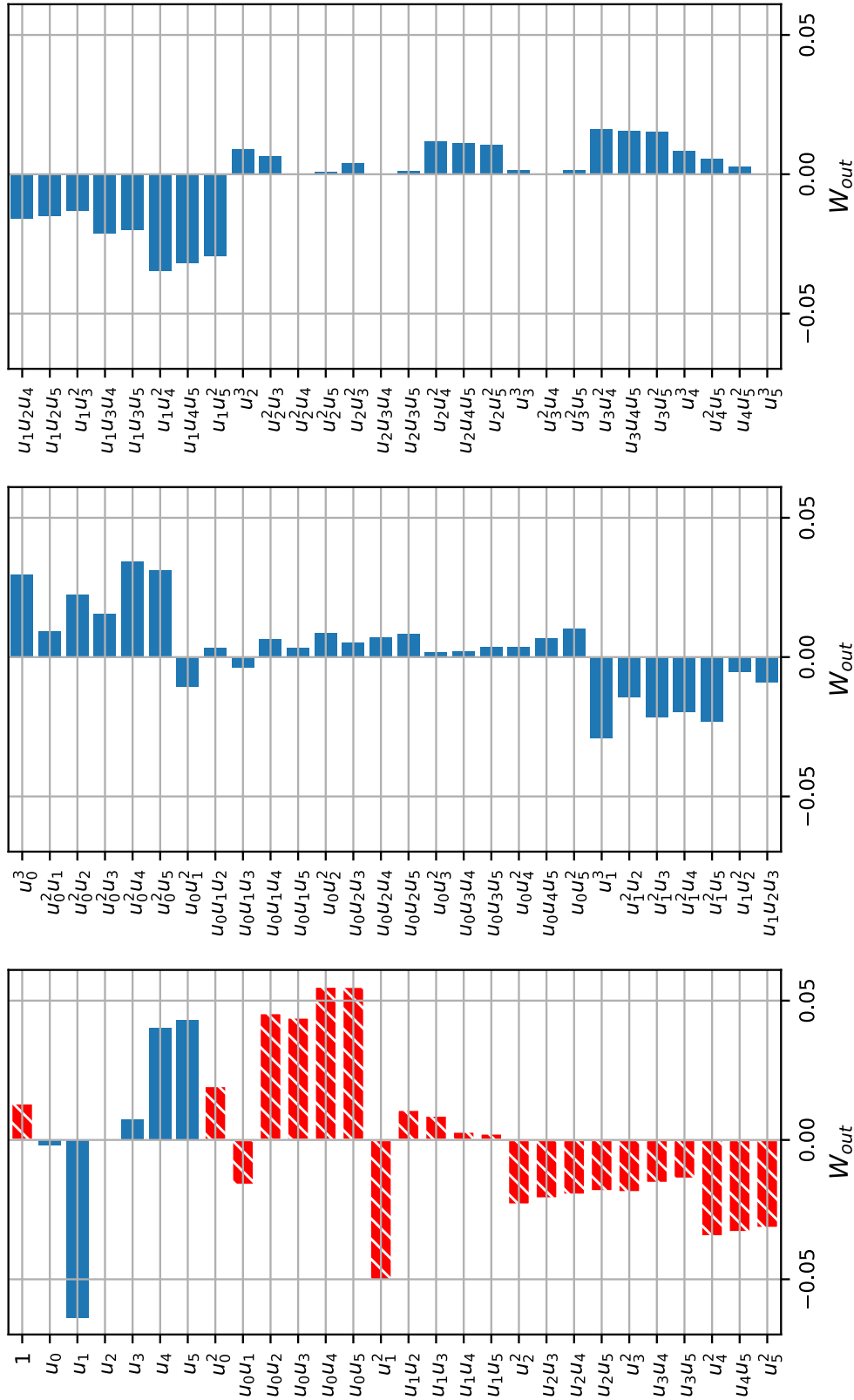


Figure 5.11: Weights in the matrix W_{out} for the NVAR trained for Mackey-Glass system forecasting. Terms that break the inversion symmetry are in dashed red. In labels, u_i is shorthand for $u(t - \tau_i \Delta t)$. As the training signal does not reflect the inversion symmetry, the red terms are not trained to near zero as they are in Fig. 5.2.

Task		W_{out}	α	$\epsilon / \tilde{\epsilon}$	Fixed Points
Lorenz '63	Inference	1×45	5.0×10^{-2}	$1.75 \pm 0.3 \times 10^{-2}$	–
Lorenz '63	Forecasting	3×28	2.5×10^{-6}	$4.51 \pm 0.9 \times 10^{-3}$	$1.2 \pm 1.4 \times 10^{-3}$
					$11.6 \pm 3.0 \times 10^{-4}$
					$7.4 \pm 1.5 \times 10^{-4}$
Double-Scroll	Forecasting	3×62	1.0×10^{-3}	$2.15 \pm 0.6 \times 10^{-2}$	$2.7 \pm 0.3 \times 10^{-3}$
Mackey-Glass	Forecasting	1×84	9.0×10^{-3}	$5.77 \pm 0.4 \times 10^{-2}$	$2.25 \pm 0.03 \times 10^{-3}$

Table 5.2: Summary of results in this chapter: the dimension of W_{out} , the ridge parameter α , the NRMSE (ϵ for inference, $\tilde{\epsilon}$ for forecasting), and the normalized distance from true fixed points to predicted fixed points.

5.5 Conclusion

In this chapter, I demonstrate the NVAR approach on two classic RC problems: state inference and system forecasting. These results are summarized in Table 5.2. I show that NVARs can solve these problems as well, or in some cases better, than the RC approach, despite the simplicity of the NVAR method. At the same time, I resolve the remaining questions raised in Chapter 4: NVARs remain useful even with only a handful of time-delay taps, and a simple extension of the nonlinear function $\mathbf{g}_n$ to include higher-order monomial terms is sufficient in many cases.

The Mackey-Glass forecasting task sits at the very edge of this simple technique. It shows that a low number of time-delay taps is enough even in systems with long, built-in time delays, as long as the taps are chosen carefully. However, it also demonstrates that a cubic nonlinearity is not enough to reproduce the precise shape of the Mackey-Glass attractor. To move forward with the NVAR technique, the nonlinear function $\mathbf{g}_n$ needs to include higher-order terms, and it needs to do so without the exponential increase in training time associated with adding more monomial terms to the output.

The Mackey-Glass system also demonstrates that symmetry matching is not necessary or even desired in all cases when using an NVAR, if the signal the NVAR is trained on never encounters that symmetry. In that case, the NVAR will utilize terms that break

the symmetry of the underlying system in order to improve forecasting accuracy within the volume of phase space that the training signal lies inside. This comes with a trade-off: prediction outside this volume of phase space, such as predicted fixed points, fails.

Unlike RCs, the weight matrix W_{out} of a trained NVAR is often interpretable. For example, it can reveal symmetries in the training data that the NVAR has learned to replicate. However, the ridge regression I use in this chapter does not encourage sparsity, and so the trained weight matrix is often dense, with many small coefficients. Using a sparse regression technique, such as Lasso regression [48], may lead to more interpretable results.

Due to their simplicity, the NVAR approach would be well-suited to hardware implementation. As an example, a hardware NVAR could be used in an engine to rapidly infer the internal, inaccessible state of the engine from external sensors. Hardware NVARs could also be used in forecasting mode as a fast signal generator that can produce a variety of signals, including many chaotic signals, only by modifying W_{out} .

Even with the unresolved questions, I find the NVAR method is very simple to implement and performs well. Due to its simplicity, I recommend attempting to use an NVAR first in all cases where an RC is considered.

Chapter 6

CONCLUSION

In this dissertation, I challenge common RC design rules, and demonstrate increasingly simple RCs that break these rules and still perform well.

In Chapter 3, I demonstrate ESN-based RCs with five classes of simple internal connectivity that all perform well on the system forecasting task. Despite the fact that recurrence is a common requirement in ESN design, two of these classes of connectivity have *no recurrence*, but continue to perform well. A network with only a single internal cycle, or no cycles at all, can perform as well as a network with many recurrent cycles. These simpler reservoir structures may benefit hardware-based RCs, where every internal connection has an associated cost.

This work reveals some new lines of possible future research. Primarily, the $\tilde{\epsilon}$ metric is a more valuable measure than ϵ_1 of how well the ESN reproduces the climate of the system, but ultimately still fails to measure attractor similarity directly. The development of such a metric would be very important to the field, as it begins to consider how well an RC reproduces climate in addition to short-term forecasting.

One of the simple connectivity classes I explore is a simple line network. The fact that this line network can perform as well as a full-connectivity network indicates that perhaps the reservoir's role is to produce a variety of time delays for the RC output layer to draw on. In Chapter 4, I summarize the NVAR method, which parallels the RC method but instead takes time-delay taps of the input directly rather than relying on a reservoir. I also sketch a new proof in the field of reservoir computing that shows a certain type of output-nonlinear

ESN is equivalent to an infinite-tap NVAR.

In Chapter 5 I build on this proof with my collaborators to implement NVARs that solve the traditional RC tasks of system state inference and system forecasting. I show that the NVAR can solve these tasks with only a few, finite taps, and that the specific nonlinearity used in the proof can be modified and extended in obvious ways to expand the number of problems it can solve. This opens the door to an entire expanding tree of interesting research. Now any task previously solved in an RC context can be attempted in an NVAR context, and the success or failure of the NVAR at solving a given problem will contribute meaningfully to both the NVAR and RC fields. In particular, the simplicity of the NVAR approach lends itself to hardware implementation, and avoids the complexities of hardware-based RCs.

Applying the NVAR approach to Mackey-Glass system forecasting reveals possibly the most important avenue for future research. Cubic terms are not sufficient for all problems, but adding higher-order monomials directly to the NVAR leads to an exponential explosion in training time. Future research will need to resolve this by including these higher-order terms without exhaustively including all monomials, and may possibly make use of nonlinear functions with infinite power series.

The NVAR is a reservoir computer pruned down to its most essential components, and now that I know it is possible to build a high-performance reservoir computer without a reservoir *at all*, I am excited to see what new directions the RC field moves towards.

REFERENCES

- [1] L. Lönnblad, C. Peterson, and T. Rönvaldsson, “Mass reconstruction with a neural network,” *Physics Letters B* **278**, 181 (1992).
- [2] I. Csabai, F. Czakó, and Z. Fodor, “Quark- and gluon-jet separation using neural networks,” *Phys. Rev. D* **44**, R1905 (1991).
- [3] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” in *Proceedings of the IEEE*, Vol. 86 (IEEE, 1998) pp. 2278–2324.
- [4] P. Y. Simard, D. Steinkraus, and J. C. Platt, “Best practices for convolutional neural networks applied to visual document analysis,” in *Proceedings of the Seventh International Conference on Document Analysis and Recognition, ICDAR '03*, Vol. 2 (IEEE, 2003) pp. 958–963.
- [5] G. Hinton, L. Deng, D. Yu, G. Dahl, A.-r. Mohamed, N. Jaitly, A. Senior, V. Vanhoucke, P. Nguyen, B. Kingsbury, and T. Sainath, “Deep neural networks for acoustic modeling in speech recognition,” *IEEE Signal Processing Magazine* **29**, 82 (2012).
- [6] A. van den Oord, S. Dieleman, H. Zen, K. Simonyan, O. Vinyals, A. Graves, N. Kalchbrenner, A. W. Senior, and K. Kavukcuoglu, “Wavenet: A generative model for raw audio,” *CoRR* (2016), arXiv:1609.03499 [cs.SD] .
- [7] A. Garcia-Pedrero and P. Gomez-Gil, “Time series forecasting using recurrent neural

- networks and wavelet reconstructed signals,” in *2010 20th International Conference on Electronics Communications and Computers (CONIELECOMP)* (2010) pp. 169–173.
- [8] Y. Bengio, P. Simard, and P. Frasconi, “Learning long-term dependencies with gradient descent is difficult,” *Trans. Neur. Netw.* **5**, 157 (1994).
 - [9] M. Lukoševičius and H. Jaeger, “Reservoir computing approaches to recurrent neural network training,” *Computer Science Review* **3**, 127 (2009).
 - [10] H. Jaeger and H. Haas, “Harnessing nonlinearity: Predicting chaotic systems and saving energy in wireless communication,” *Science* **304**, 78 (2004).
 - [11] J. Pathak, Z. Lu, B. R. Hunt, M. Girvan, and E. Ott, “Using machine learning to replicate chaotic attractors and calculate Lyapunov exponents from data,” *Chaos: An Interdisciplinary Journal of Nonlinear Science* **27**, 121102 (2017).
 - [12] Z. Lu, J. Pathak, B. Hunt, M. Girvan, R. Brockett, and E. Ott, “Reservoir observers: Model-free inference of unmeasured variables in chaotic systems,” *Chaos: An Interdisciplinary Journal of Nonlinear Science* **27**, 041102 (2017).
 - [13] A. Haluszczyński and C. R  th, “Good and bad predictions: Assessing and improving the replication of chaotic attractors by means of reservoir computing,” *Chaos: An Interdisciplinary Journal of Nonlinear Science* **29**, 103143 (2019).
 - [14] D. Canaday, A. Griffith, and D. J. Gauthier, “Rapid time series prediction with a hardware-based reservoir computer,” *Chaos: An Interdisciplinary Journal of Nonlinear Science* **28**, 123119 (2018).
 - [15] P. Antonik, M. Hermans, F. Duport, M. Haelterman, and S. Massar, “Towards pattern generation and chaotic series prediction with photonic reservoir computers,” in *Proceedings of SPIE*, Vol. 9732 (2016) p. 97320B.
 - [16] H. Jaeger, “The “echo state” approach to analysing and training recurrent neural networks-with an erratum note,” Bonn, Germany: German National Research Center for Information Technology GMD Technical Report **148**, 13 (2001).

- [17] M. Lukoševičius, “A practical guide to applying echo state networks,” in *Neural Networks: Tricks of the Trade: Second Edition*, edited by G. Montavon, G. B. Orr, and K.-R. Müller (Springer Berlin Heidelberg, Berlin, Heidelberg, 2012) pp. 659–686.
- [18] A. Griffith, A. Pomerance, and D. J. Gauthier, “Forecasting chaotic systems with very low connectivity reservoir computers,” *Chaos: An Interdisciplinary Journal of Nonlinear Science* **29**, 123108 (2019).
- [19] D. J. Gauthier, E. Bollt, A. Griffith, and W. A. S. Barbosa, “Next generation reservoir computing,” (2021), arXiv:2106.07688 [cs.LG] .
- [20] N. D. Haynes, M. C. Soriano, D. P. Rosin, I. Fischer, and D. J. Gauthier, “Reservoir computing with a single time-delay autonomous Boolean node,” *Phys. Rev. E* **91**, 020801 (2015).
- [21] A. Rodan and P. Tiño, “Minimum complexity echo state network,” *IEEE Transactions on Neural Networks* **22**, 131 (2011).
- [22] H. Jaeger, “Short term memory in echo state networks,” Bonn, Germany: German National Research Center for Information Technology GMD Technical Report **152** (2002).
- [23] Z. Lu, B. R. Hunt, and E. Ott, “Attractor reconstruction by machine learning,” *Chaos: An Interdisciplinary Journal of Nonlinear Science* **28**, 061104 (2018).
- [24] T. Lymburn, D. M. Walker, M. Small, and T. Jüngling, “The reservoir’s perspective on generalized synchronization,” *Chaos: An Interdisciplinary Journal of Nonlinear Science* **29**, 093133 (2019).
- [25] Z. Lu and D. S. Bassett, “Invertible generalized synchronization: A putative mechanism for implicit learning in neural systems,” *Chaos: An Interdisciplinary Journal of Nonlinear Science* **30**, 063133 (2020).

- [26] H. Jaeger, M. Lukoševičius, D. Popovici, and U. Siewert, “Optimization and applications of echo state networks with leaky- integrator neurons,” *Neural Networks* **20**, 335 (2007).
- [27] J. Yperman and T. Becker, “Bayesian optimization of hyper-parameters in reservoir computing,” *CoRR* (2016), arXiv:1611.05193 [cs.LG] .
- [28] J. R. Maat, N. Gianniotis, and P. Protopapas, “Efficient optimization of echo state networks for time series datasets,” in *International Joint Conference on Neural Networks (IJCNN)* (2018).
- [29] The scikit-optimize contributors, “scikit-optimize/scikit-optimize: v0.5.2,” (2018).
- [30] J. Pathak, B. Hunt, M. Girvan, Z. Lu, and E. Ott, “Model-free prediction of large spatiotemporally chaotic systems from data: A reservoir computing approach,” *Phys. Rev. Lett.* **120**, 024102 (2018).
- [31] W. A. S. Barbosa, A. Griffith, G. E. Rowlands, L. C. G. Govia, G. J. Ribeill, M.-H. Nguyen, T. A. Ohki, and D. J. Gauthier, “Symmetry-aware reservoir computing,” (2021), arXiv:2102.00310 [cs.NE] .
- [32] J. Herteux and C. R  th, “Breaking symmetries of the reservoir equations in echo state networks,” *Chaos: An Interdisciplinary Journal of Nonlinear Science* **30**, 123142 (2020).
- [33] D. Viswanath, *Lyapunov exponents from random Fibonacci sequences to the Lorenz equations*, Ph.D. thesis, Cornell University (1998).
- [34] J. R. Dormand and P. J. Prince, “A family of embedded Runge-Kutta formulae,” *Journal of Computational and Applied Mathematics* **6**, 19 (1980).
- [35] R. Rifkin and R. Lippert, *Notes on Regularized Least Squares*, Tech. Rep. MIT-CSAIL-TR-2007-025 (Massachusetts Institute of Technology Computer Science and Artificial Intelligence Laboratory, 2007).

- [36] D. W. Scott, *Multivariate Density Estimation: Theory, Practice, and Visualization*, 1st ed. (Wiley, Berlin, Heidelberg, 1992).
- [37] R. Ishar, E. Kaiser, M. Morzyński, D. Fernex, R. Semaan, M. Albers, P. S. Meysonnat, W. Schröder, and B. R. Noack, “Metric for attractor overlap,” *Journal of Fluid Mechanics* **874**, 720–755 (2019).
- [38] H. T. Siegelmann, B. G. Horne, and C. L. Giles, “Computational capabilities of recurrent NARX neural networks,” *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)* **27**, 208 (1997).
- [39] E. Bollt, “On explaining the surprising success of reservoir computing forecaster of chaos? The universal machine learning dynamical system with contrast to VAR and DMD,” *Chaos: An Interdisciplinary Journal of Nonlinear Science* **31**, 013108 (2021).
- [40] F. Takens, in *Dynamical Systems and Turbulence, Warwick 1980: Proceedings of a Symposium Held at the University of Warwick 1979/80*, edited by D. Rand and L.-S. Young (Springer Berlin Heidelberg, Berlin, Heidelberg, 1981) pp. 366–381.
- [41] E. N. Lorenz, “Deterministic nonperiodic flow,” *Journal of the Atmospheric Sciences* **20**, 130 (1963).
- [42] P. Dierckx, *Curve and surface fitting with splines* (Clarendon Press, Oxford, 1995).
- [43] S. Krishnagopal, G. Katz, M. Girvan, and J. Reggia, “Encoding of a chaotic attractor in a reservoir computer: A directional fiber investigation,” in *2019 International Joint Conference on Neural Networks (IJCNN)* (2019) pp. 1–8.
- [44] J. J. More, B. S. Garbow, and K. E. Hillstom, *User guide for MINPACK-1. [In FORTRAN]*, Tech. Rep. (Argonne National Laboratory, 1980).
- [45] M. J. D. Powell, “A hybrid method for nonlinear equations,” in *Numerical Methods for Nonlinear Algebraic Equations*, edited by P. Rabinowitz (Gordon and Breach, 1970).

- [46] M. O. Franz and B. Schölkopf, “A unifying view of Wiener and Volterra theory and polynomial kernel regression,” *Neural Computation* **18**, 3097 (2006).
- [47] G. Ansmann, “Efficiently and easily integrating differential equations with JiTCODE, JiTCDDE, and JiTCSDE,” *Chaos: An Interdisciplinary Journal of Nonlinear Science* **28**, 043116 (2018).
- [48] R. Tibshirani, “Regression shrinkage and selection via the lasso,” *Journal of the Royal Statistical Society: Series B (Methodological)* **58**, 267 (1996).
- [49] O. E. Rössler, “An equation for continuous chaos,” *Physics Letters A* **57**, 397 (1976).
- [50] D. J. Gauthier and J. C. Bienfang, “Intermittent loss of synchronization in coupled chaotic oscillators: Toward a new criterion for high-quality synchronization,” *Phys. Rev. Lett.* **77**, 1751 (1996).
- [51] M. C. Mackey and L. Glass, “Oscillation and chaos in physiological control systems,” *Science* **197**, 287 (1977).

Appendix A

EXAMPLE SYSTEMS

Reservoir computers are most useful as tools for answering questions about dynamic systems, such as state inference and forecasting. Throughout this dissertation, I use many small chaotic systems for example RC tasks. For reference, they are included here.

A.1 Lorenz '63

The Lorenz '63 chaotic system is described by

$$\begin{aligned}\dot{x} &= \sigma(y - x), \\ \dot{y} &= x(\rho - z) - y, \\ \dot{z} &= xy - \beta z,\end{aligned}\tag{A.1}$$

with standard parameters $\sigma = 10$, $\rho = 28$, and $\beta = 8/3$ [41]. The attractor of this system in Fig. A.1 can be visualized easily in two dimensions by projecting the three-dimensional trajectory of the system onto a plane. I show the attractor in the x/z plane in Fig. A.2, and I use this projection throughout this dissertation whenever I compare ESN or NVAR output to the true Lorenz attractor.

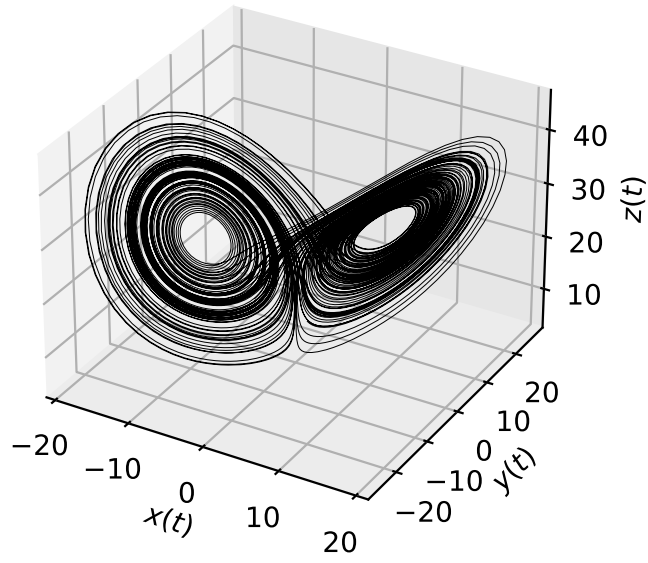


Figure A.1: The true Lorenz attractor, produced by integrating Eq. (A.1).

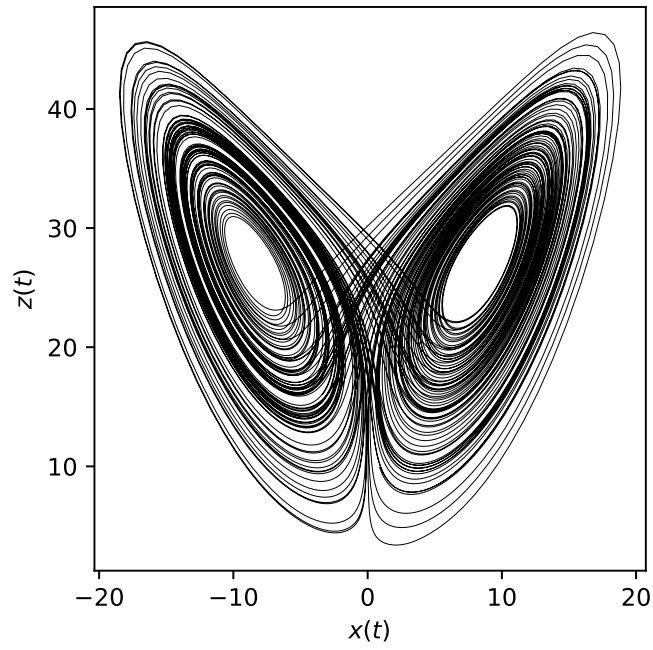


Figure A.2: The true Lorenz attractor projected on to the x/z plane.

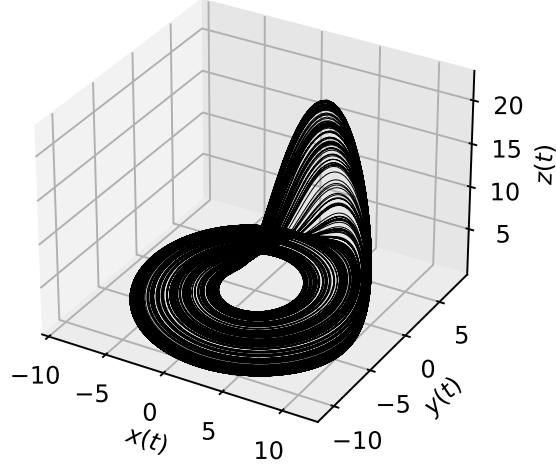


Figure A.3: The true Rössler attractor, produced by integrating Eq. (A.2).

A.2 Rössler

The Rössler system is described by

$$\begin{aligned}\dot{x} &= -y - z, \\ \dot{y} &= x + ay, \\ \dot{z} &= b + z(x - c),\end{aligned}\tag{A.2}$$

with standard parameters $a = 0.2$, $b = 0.2$, $c = 5.7$ [49]. The attractor of this system is visualized in Fig. A.3.

The z component of this system mostly stays near zero, with rare positive spikes. This makes prediction with an ESN difficult. To make this component of the system more suitable for ESN prediction, I use $\log z$ instead for both ESN input and prediction output.

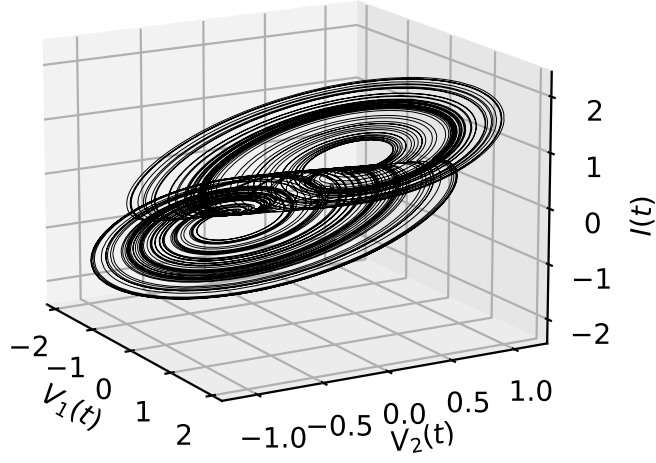


Figure A.4: The true double-scroll circuit attractor, produced by integrating Eq. (A.3).

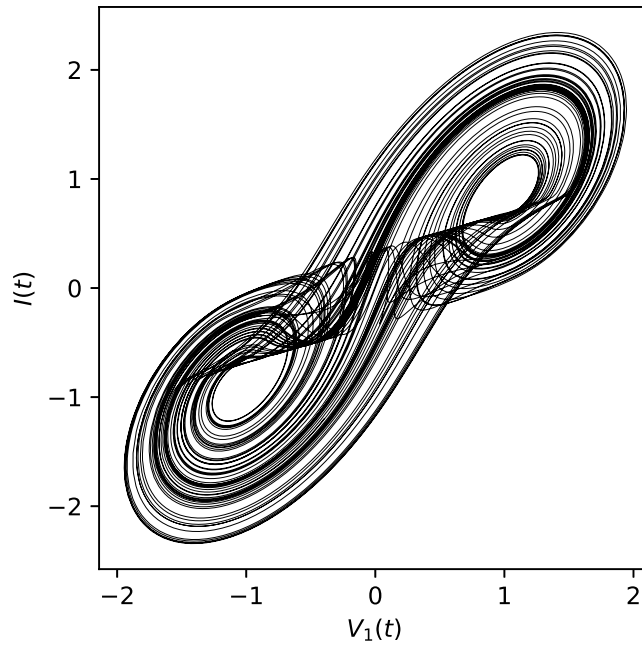


Figure A.5: The true double-scroll circuit attractor projected on to the V_1/I plane.

A.3 Double-Scroll

The double-scroll chaotic circuit is described by the dimensionless equations

$$\begin{aligned}\dot{V}_1 &= \frac{V_1}{R_1} - \frac{V_1 - V_2}{R_2} - 2I_r \sinh(\alpha(V_1 - V_2)), \\ \dot{V}_2 &= \frac{V_1 - V_2}{R_2} + 2I_r \sinh(\alpha(V_1 - V_2)) - I, \\ \dot{I} &= V_2 - R_4 I,\end{aligned}\tag{A.3}$$

with parameters $R_1 = 1.2$, $R_2 = 3.44$, $R_4 = 0.193$, $I_r = 2.25 \times 10^{-5}$, and $\alpha = 11.6$ [50]. The attractor of this system is depicted in Fig. A.4. As with the Lorenz system, for comparison purposes I will use a projection of the attractor on to the V_1/I plane, depicted in Fig. A.5.

A.4 Mackey-Glass

The Mackey-Glass system is described by the time-delay differential equation

$$\dot{u}(t) = \beta \frac{u(t - \tau)}{1 + u(t - \tau)^n} - \gamma u(t),\tag{A.4}$$

with standard parameters $\beta = 0.2$, $\gamma = 0.1$, $\tau = 17$, $n = 10$ [51]. The attractor of this system is most commonly visualized as a time-delay embedding of the trajectory, using τ as the delay parameter, as depicted in Fig. A.6.

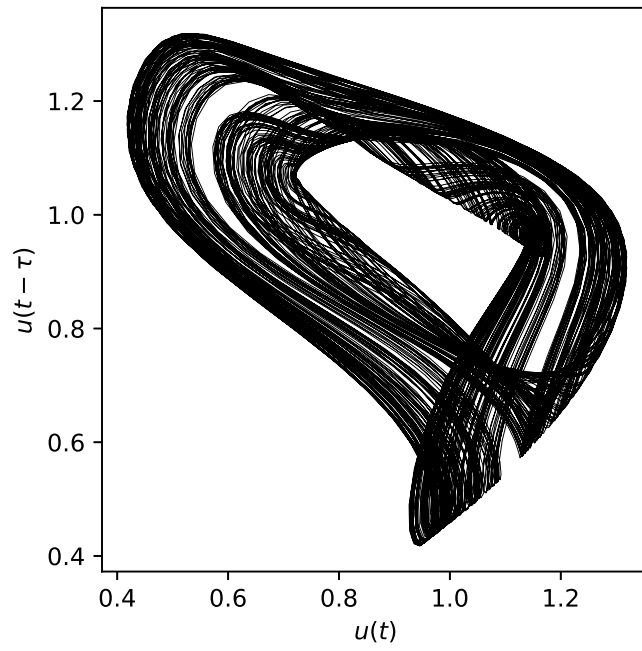


Figure A.6: The true Mackey-Glass attractor represented as a time delay embedding using τ as the delay, produced by integrating Eq. (A.4).